

گزارش کامل و مرحله به مرحله عملکرد کد

سیستم معامله گر هوش مصنوعی

قراردادهای آتی دائمی USDT-M صرافی بایننس — تایم فریم ۵ دقیقه

۱۰ قانون تصمیم گیری

۴ مدل یادگیری ماشین

۶ مازول (A تا F)

۱۴۳۲۶ خط کد پایتون

۳۹ فایل

مخزن: mtmtest2/secret.binance.pj

شاخه: claude/code-operation-detailed-report-zwztu2

این گزارش طوری نوشته شده که بدون هیچ دانش قبلی هم کاملاً قابل فهم باشد.

۰	این پروژه اصلاً چیست؟ (با یک مثال ساده)
۱	نقشه کلی سیستم — شش کارخانه پشت سر هم
۲	ساختار پوشه‌ها و فایل‌ها — چه کسی چه کاره است
۳	اتاق تنظیمات — config/settings.py
۴	ابزارهای پایه — پوشه core/
۵	لحظه روشن شدن سیستم — main.py
۶	مرحله ۱: انتخاب ارزها (universe.py)
۷	مرحله ۲: جمع‌آوری داده از بایننس (fetcher.py)
۸	مرحله ۳: پلیس کیفیت داده (qc_validator.py)
۹	مرحله ۴: انبار داده — پایگاه داده SQLite
۱۰	خط لوله مازول (A pipeline.py) — پرکردن تاریخچه و چرخه زنده
۱۱	مرحله ۵: ساخت ویژگی‌ها (features.py و indicators.py)
۱۲	مرحله ۶: معلم مدل — برجسب‌زنی (labeler.py)
۱۳	مرحله ۷: ساخت دفترچه تمرین (processor.py)
۱۴	مرحله ۸: چهار مغز کوچک — مدل‌های یادگیری ماشین (ml_models.py)
۱۵	قراردادهای بین مازول‌ها (schemas.py)
۱۶	مرحله ۹: قاضی نهایی — موتور تصمیم و ۱۰ قانون (decision_engine.py)
۱۷	مرحله ۱۰: اجرای معامله — کاغذی و واقعی
۱۸	مرحله ۱۱: نگهبان ریسک و کلید فرمز (risk_guard.py)
۱۹	مرحله ۱۲: دفتر خاطرات سیستم (audit_engine.py)
۲۰	مرحله ۱۳: پنل وب و کنترلرها (web_app.py)
۲۱	بک‌تست — امتحان روی گذشته (backtester.py)
۲۲	یک چرخه ۵ دقیقه‌ای کامل، ثانیه به ثانیه
۲۳	چرخه عمر سیستم و فازها
۲۴	دستورهای خط فرمان
۲۵	جدول‌های پایگاه داده
۲۶	لایه‌های ایمنی — کجا جلوی فاجعه گرفته می‌شود
۲۷	محدودیت‌ها و نقاط ضعف واقعی
۲۸	واژه‌نامه

۰۰. این پروژه اصلاً چیست؟

این پروژه یک ربات معامله‌گر خودکار است. کارش این است که هر ۵ دقیقه یک‌بار به بازار ارز دیجیتال (بخش «قراردادهای آتی دائمی» صرافی بایننس) نگاه کند، فکر کند، و تصمیم بگیرد که «الان بخرم؟ بفروشم؟ یا دست نگه دارم؟».

👉 **به زبان بچه‌گانه:** فرض کن یک آشپزخانه خیلی بزرگ داری. هر ۵ دقیقه یک جعبه مواد اولیه از بازار می‌رسد. یک نفر مواد را بازرسی می‌کند که خراب نباشند، یک نفر آن‌ها را خرد و آماده می‌کند، چهار سرآشپز هر کدام یک نظر می‌دهند، یک مدیر همه نظرها را می‌شنود و تصمیم نهایی را می‌گیرد، یک نفر غذا را سرو می‌کند، و یک نگهبان همیشه کنار در ایستاده تا اگر آتش‌سوزی شد همه‌چیز را خاموش کند. این پروژه دقیقاً همین آشپزخانه است — فقط به جای غذا، «معامله» تولید می‌کند.

۰۰۱. چند تعریف پایه که باید بدانی

اصطلاح	یعنی چه؟
کندل (Candle)	خلاصه قیمت در یک بازه زمانی. هر کندل ۵ دقیقه‌ای ۵ عدد دارد: قیمت باز شدن (open)، بالاترین قیمت (high)، پایین‌ترین قیمت (low)، قیمت بسته شدن (close) و حجم معاملات (volume).
لانگ (Long)	شرط‌بندی روی بالا رفتن قیمت. می‌خری، گران‌تر می‌فروشی.
شورت (Short)	شرط‌بندی روی پایین آمدن قیمت. در بازار آتی می‌توانی چیزی را که نداری بفروشی و بعداً ارزان‌تر پس بخری.
اهرم (Leverage)	وام گرفتن از صرافی. با ۱۰۰ دلار و اهرم ۵، مثل این است که ۵۰۰ دلار معامله می‌کنی. سود ۵ برابر، ضرر هم ۵ برابر. در این سیستم سقف اهرم ۱۰ است.
حد سود (TP)	قیمتی که اگر بازار به آن رسید، معامله را با سود می‌بندیم.
حد ضرر (SL)	قیمتی که اگر بازار به آن رسید، معامله را با ضرر کم می‌بندیم تا ضرر بزرگ نشود.
لیکوئید شدن	وقتی ضرر آن‌قدر زیاد شود که پول ودیعه تو تمام شود، صرافی به‌زور معامله را می‌بندد و کل آن پول از بین می‌رود.
فاندینگ (Funding)	در بازار آتی دائمی، هر ۸ ساعت یک‌بار یک طرف (لانگ‌ها یا شورت‌ها) به طرف دیگر مبلغ کوچکی می‌پردازد. این یک هزینه واقعی است.
اسلیپیج (Slippage)	فرق بین قیمتی که انتظار داشتی و قیمتی که واقعاً معامله شد. همیشه به ضرر توست.
کارمزد (Fee)	سهم صرافی از هر معامله. اینجا ۰.۰۵٪ برای سفارش بازار (taker) و ۰.۰۲٪ برای سفارش محدود (maker).

۰۰۲. سه جمله مهم درباره فلسفه این کد

۱ **هیچ‌وقت روی داده خراب معامله نکن.** اگر داده یک ارز مشکوک بود، سیستم آن ارز را برای آن چرخه کاملاً کنار می‌گذارد. هرگز عدد جعلی نمی‌سازد.

۲ **هیچ‌وقت از آینده خبر نداشته باش.** همه محاسبات فقط از داده‌های «تا همین لحظه» استفاده می‌کنند. این را در متن کد «علی بودن» یا causality می‌گویند و مهم‌ترین قانون کل پروژه است.

۳ **هیچ‌وقت خودت شروع نکن.** ربات تا وقتی که آدم دکمه «شروع» را نزند، حتی یک معامله هم انجام نمی‌دهد. فقط داده جمع می‌کند و آماده می‌ماند.

⚠ **هشدار:** این نرم‌افزار تضمین سودآوری ندارد. خود فایل README پروژه می‌گوید در تست خارج-از-نمونه (روی داده ندیده) نرخ برد ۰٪ بوده است. مدل‌ها همراه پروژه نمی‌آیند؛ تو باید خودت روی داده خودت آموزششان بدهی. معامله اهرمی می‌تواند کل سرمایه را نابود کند.

۱. نقشه کلی سیستم — شش کارخانه پشت سر هم

کل پروژه به شش بخش تقسیم شده که به آن‌ها «ماژول» می‌گوییم و با حروف **A** تا **F** نام‌گذاری شده‌اند. داده مثل یک نوار نقاله از **A** شروع می‌شود و تا **E** می‌رود، و **F** همه مسیر را تماشا و ثبت می‌کند.

```
+-----+
| main.py = زمان‌بند ۵ دقیقه‌ای |
+-----+
| (asyncio یک حلقه رویداد) |
+-----+ +-----+ +-----+ +-----+ +-----+
| ماژول A |-->| ماژول B |-->| ماژول C |-->| ماژول D |-->| ماژول E |
| داده QC+ | | ویژگی + | | مدل ۴ | | موتور | | اجرا + |
| | | برچسب | | ML | | تصمیم | | ریسک |
+-----+ +-----+ +-----+ +-----+ +-----+
| | |
+-----> SQLite <-----+
      ^
+-----+
| ثبت وقایع F: ماژول |
| پنل وب + |
+-----+
```

ماژول	پوشه	مسئولیت (به زبان ساده)
A	module_a_data/	«مأمور خرید و بازرش». از بایننس داده می‌گیرد، کیفیتش را چک می‌کند، خرابی‌ها را تعمیر می‌کند و در انبار (پایگاه داده) می‌گذارد.
B	module_b_features/	«آشپز آماده‌سازی». از قیمت خام، ۲۶ عدد معنی‌دار می‌سازد (روند، نوسان، مومنتوم، رژیم بازار) و برای آموزش، جواب‌های درست را هم تولید می‌کند.
C	module_c_ml/ml_models.py	«چهار سرآشپز». چهار مدل مستقل که هرکدام فقط به یک سؤال جواب می‌دهند: کجا؟ کی؟ چگونه؟ چقدر؟
D	module_c_ml/decision_engine.py	«مدیر/قاضی». هیچ هوش مصنوعی ندارد. فقط ۱۰ قانون سفت‌وسخت را یکی‌یکی چک می‌کند. اولین قانونی که رد شود، کار تمام است.
E	module_e_execution/	«پیشخدمت + نگهبان». سفارش را به صرافی می‌فرستد (یا شبیه‌سازی می‌کند)، و نگهبان ریسک می‌تواند همه چیز را متوقف کند.
F	module_f_panel/	«دفتردار + اتاق کنترل». هر تصمیم را ثبت می‌کند و یک صفحه وب می‌سازد که تو با آن سیستم را می‌بینی و کنترل می‌کنی.

نکته: ماژول‌ها فقط از طریق «قراردادهای اعتبارسنجی‌شده» با هم حرف می‌زنند (کلاس‌های Pydantic در فایل schemas.py). یعنی هیچ ماژولی نمی‌تواند به دیگری داده‌ی بی‌معنی بدهد — چون در همان لحظه ساخت داده، قوانین بررسی می‌شوند و اگر خطا باشد برنامه فریاد می‌زند.

۲. ساختار پوشه‌ها و فایل‌ها — چه کسی چه‌کاره است

پروژه ۳۹ فایل دارد. جدول زیر همه آن‌ها را با تعداد خط و وظیفه فهرست می‌کند.

ریشه پروژه

فایل	خط	وظیفه
main.py	۱۱۳۶	ارکستر اصلی. همه مازول‌ها را می‌سازد، زمان‌بند ۵ دقیقه‌ای و سرور وب را اجرا می‌کند، و چرخه معاملات را می‌گرداند. کلاس اصلی: TradingSystem.
requirements.txt	۳۸	فهرست کتابخانه‌های لازم: ccxt, pandas, numpy, lightgbm, xgboost, hmmlearn, .arch, fastapi, uvicorn, SQLAlchemy, aioredis, APScheduler.
.env.example	—	نمونه فایل تنظیمات. کپی می‌کنی به .env، و مقادیر را عوض می‌کنی.
.gitignore	—	می‌گوید کدام فایل‌ها در گیت ذخیره نشوند: *.db, *.joblib, data/, logs/, artifacts/, .env — یعنی پایگاه داده، مدل‌ها و کلیدهای API هرگز روی گیت‌هاب نمی‌روند.
README.md	—	راهنمای انگلیسی پروژه.

پوشه config/ — تنظیمات

فایل	خط	وظیفه
settings.py	۴۲۴	تمام عده‌های قابل تنظیم پروژه در ۱۲ گروه. هیچ فایل دیگری اجازه ندارد عدد جادویی داشته باشد.
__init__.py	۷	فقط برای شناسایی پوشه به‌عنوان پکیج.

پوشه core/ — ابزارهای مشترک

فایل	خط	وظیفه
exceptions.py	۷۳	۱۲ نوع خطای اختصاصی (خطای داده، خطای مدل، خطای اجرا و ...).
logger.py	۱۰۳	سیستم ثبت لاگ + یک حافظه حلقوی ۸۰۰ خطی که پل وب از آن می‌خواند.
utils.py	۱۹۲	توابع کوچک: کار با زمان، عدد امن، تأخیر نمایی برای تلاش مجدد.

مازول A — module_a_data/

فایل	خط	وظیفه
models.py	۳۲۰	قالب‌های سختگیرانه داده: کندل، اسنپ‌شات دفتر سفارش، متریک‌های آتی، گزارش کیفیت.
fetcher.py	۵۵۸	ارتباط با بایننس از طریق کتابخانه CCXT با تلاش مجدد و محدودکننده همزمانی.
qc_validator.py	۷۲۴	پلیس کیفیت. ۴ خانواده بررسی + تعمیر خودکار داده.
db_models.py	۲۳۴	تعریف ۶ جدول پایگاه داده.
db_handler.py	۶۷۴	همه عملیات خواندن/نوشتن پایگاه داده، کاملاً غیرهمزمان.
pipeline.py	۳۲۵	همان‌گونه مازول A: گرفتن ← اعتبارسنجی ← تعمیر ← ذخیره.
universe.py	۴۸۸	کشف و غربال ارزها + ذخیره انتخاب کاربر.

B — module_b_features/ مازول

فایل	خط	وظیفه
indicators.py	۲۶۴	اندیکاتورهای پایه از صفر نوشته شده: ATR, RSI, KAMA, FDI, ADX, Bollinger, رتبه درصدی, Z-score, شیب.
features.py	۹۸۹	ساخت ۲۶ ویژگی نهایی + مدل های GARCH و HMM.
labeler.py	۶۱۸	«معلم»: با نگاه به آینده جواب درست را می سازد (فقط برای آموزش).
processor.py	۴۱۶	ترکیب همه چیز در یک دیتاست تمیز برای آموزش، یا یک ردیف برای پیش بینی زنده.

D — module_c_ml/ و مازول C

فایل	خط	وظیفه
schemas.py	۳۳۵	قراردادهای بین ماژول ها. «کمبریک» سیستم.
ml_models.py	۱۱۳۲	چهار مدل + زیرسیستم مدیریت آن ها (آموزش، ذخیره، بارگذاری، پیش بینی).
decision_engine.py	۵۵۷	موتور تصمیم با ۱۰ قانون. تنها تولیدکننده سیگنال معاملاتی.

E — module_e_execution/ مازول

فایل	خط	وظیفه
models.py	۳۵۹	ساختار «موقعیت باز»، وضعیت حساب، گزارش اجرا. ریاضیات سود/زیان اینجاست.
risk_guard.py	۴۲۴	نگهبان ریسک: ماشین حالت سبز/زرد/قرمز + کلید قرمز.
executor.py	۸۳۹	اجرای واقعی روی بایننس با پول واقعی.
paper_trader.py	۵۵۵	اجرای کاغذی (شبیه سازی) با همان رابط دقیق.
backtester.py	۶۶۴	بخش مجدد تاریخچه برای سنجش عملکرد.

F — module_f_panel/ مازول

فایل	خط	وظیفه
audit_engine.py	۳۷۵	ثبت هر تصمیم (حتی «معامله نکن») در پایگاه داده به صورت دسته ای.
setup_state.py	۱۳۲	تعریف فازهای سیستم و نوار پیشرفت راه اندازی.
web_app.py	۴۴۶	سرور ۴ FastAPI: صفحه HTML + حدود ۱۵ نقطه API.
templates.py	۷۹۲	قالب های HTML/JS پتل، همه داخل خود فایل پاتون (بدون پوشه static).

۳. اتاق تنظیمات — config/settings.py

👤 **به زبان بچه‌گانه:** این فایل مثل «تابلوی کلیدهای برق» یک ساختمان است. همهٔ کلیدها اینجا جمع شده‌اند. اگر بخواهی چیزی را عوض کنی، فقط اینجا (یا در فایل env) دست می‌زنی — نه در دل کد.

۳-۱. تنظیمات از کجا خوانده می‌شود؟

سه لایه، از ضعیف به قوی:

- مقدارهای پیش‌فرض که در خود همین فایل نوشته شده‌اند.
 - فایل env، در ریشهٔ پروژه.
 - متغیرهای محیطی واقعی سیستم عامل (قوی‌ترین).
- برای تنظیمات تودرتو از دو زیرخط استفاده می‌شود. مثلاً برای تغییر حداقل اطمینان جهت:

```
DECISION_MIN_DIRECTION_CONFIDENCE=0.72
RISK_MAX_LEVERAGE=5
```

۳-۲. دوازده گروه تنظیمات

۱) ExchangeSettings — اتصال به صرافی

کلید	پیش‌فرض	معنی
api_key / api_secret	خالی	کلیدهای بایننس. برای کاغذی لازم نیست.
testnet	true	true یعنی شبکهٔ آزمایشی (پول تقلبی). برای پول واقعی باید false شود.
request_timeout_ms	۲۰۰۰	حداکثر ۲۰ ثانیه انتظار برای هر درخواست.
max_retries	۵	هر درخواست ناموفق تا ۵ بار تکرار می‌شود.
backoff_base/max_seconds	۶۰ / ۱	تأخیر بین تلاش‌ها: ۱، ۲، ۴، ۸ ثانیه... تا سقف ۶۰.
backoff_jitter	۰.۲۵	±۲۵٪ نویز تصادفی روی تأخیر، تا همهٔ درخواست‌ها همزمان دوباره نروند.
max_concurrent_requests	۸	حداکثر ۸ درخواست همزمان — تا بایننس ما را بلاک نکند.

۲) DataSettings — داده

کلید	پیش‌فرض	معنی
timeframe	5m	تایم‌فریم ثابت ۵ دقیقه.
timeframe_ms	۳۰۰۰۰	۵ دقیقه به میلی‌ثانیه. کل سیستم با میلی‌ثانیه کار می‌کند.
ohlcv_limit	۱۵۰۰	حداکثر کندل در هر درخواست (سقف خود بایننس).
history_bootstrap_candles	۲۱۰۲۴۰	عمق تاریخچه = ۲ سال (۲ × ۳۶۵ × ۲۴ × ۶۰ = ۲۱۰۲۴۰ کندل).
drop_unclosed_candle	true	کندل نیمه‌تمام حذف می‌شود؛ فقط کندل‌های بسته‌شده معتبرند.
cycle_second_offset	۱۰	چرخه در ثانیه ۱۰ هر ۵ دقیقه اجرا می‌شود، نه ثانیهٔ صفر.

❓ **چرا این‌طوری؟** چرا ثانیه ۱۰ و نه صفر؟ چون درست در ثانیه ۰۰:۰۰ خود بایننس هنوز مشغول بستن کندل است. اگر دقیقاً همان لحظه بررسی، آخرین کندل را به تو نمی‌دهد و یک ردیف کم می‌آوری. ۱۰ ثانیه صبر کردن این مشکل را کاملاً حل می‌کند.

۳) UniverseSettings – غربال ارزها

کلید	پیش فرض	معنی
target_count	۳۰	دکمه «پیشنهاد» چند ارز را تیک بزند.
min_quote_volume_24h	۵۰ میلیون	حداقل حجم معاملات ۲۴ ساعته به USD.
max_spread_bps	۶	حداکثر فاصله خرید/فروش، برحسب «پایه» (۶ پایه = ۰.۰۶٪).
min_history_days	۹۰	ارز باید حداقل ۹۰ روز از لیست شدنش گذشته باشد.
reference_equity	۱۰۰۰	اندازه حساب مرجع برای غربال «تناسب با سرمایه کوچک».
max_granularity_fraction	۰.۲۵	هزینه یک «پله» از مقدار سفارش نباید از ۲۵٪ کوچکترین پوزیشن بیشتر باشد.
discovery_cache_seconds	۹۰۰	لیست ارزها ۱۵ دقیقه کش می‌شود.
weight_*	۰.۲۰ / ۰.۲۵ / ۰.۴۵ ۰.۱۰ /	وزن نقدشوندگی، اسپرد، مقرون به صرفه بودن و تاریخچه در امتیازدهی.

۴) QCSettings – کنترل کیفیت

کلید	پیش فرض	معنی
max_heal_attempts	۴	حداکثر ۴ بار تلاش برای تعمیر داده خراب.
heal_backoff_seconds	۲	تأخیر پایه بین تلاش‌های تعمیر.
volume_spike_median_multiple	۵۰	حجم بیش از ۵۰ برابر میانه = مشکوک.
return_mad_zscore_limit	۱۴	سقف Z-SCORE مقاوم برای بازدهی.
max_abs_candle_return	۰.۳۵	هیچ کندل ۵ دقیقه‌ای نباید بیش از ۳۵٪ حرکت کند.
max_zero_volume_ratio	۰.۲۵	بیش از ۲۵٪ کندل با حجم صفر = فید مرده.
max_lagBars	۳	آخرین کندل نباید بیش از ۳ کندل عقب باشد.
min_rows_for_statistics	۳۰	زیر ۳۰ ردیف، آزمون‌های آماری اجرا نمی‌شوند.

۵) FeatureSettings – پارامترهای ویژگی‌سازی

کلید	پیش فرض	معنی
kama_window / fast / slow	۳۰ / ۲ / ۱۰	پنجره و سرعت‌های میانگین متحرک تطبیقی.
fdi_window	۳۰	پنجره بُعد فراکتالی.
atr / rsi / adx_window	۱۴	پنجره‌های کلاسیک ۱۴ کندلی.
garch_window / refit_every	۵۰ / ۵۰۰	GARCH روی ۵۰۰ کندل اخیر، هر ۵۰ کندل دوباره برازش می‌شود.
hmm_states / window / refit_every	۱۰۰ / ۱۰۰۰ / ۴	۴ رژیم بازار، پنجره ۱۰۰۰ کندلی، برازش مجدد هر ۱۰۰ کندل.
hmm_seed	۴۲	بذر تصادفی ثابت ← نتیجه همیشه تکرارپذیر است.
rank_window	۲۸۸	۲۸۸ کندل ۵ دقیقه‌ای = دقیقاً ۲۴ ساعت.
max_feature_workers	۴	حداکثر ۴ ارز همزمان ویژگی‌سازی شوند.

۶) LabelSettings – برچسب‌زنی

کلید	پیش فرض	معنی
------	---------	------

tp_atr_multiple	۲۰	حد سود = ۲ برابر ATR.
sl_atr_multiple	۱۰	حد ضرر = ۱ برابر ATR. یعنی نسبت پاداش/ریسک هدف ۲:۱.
max_holding_bars	۴۸	حداکثر ۴۸ کندل = ۴ ساعت نگهداری.
low/medium/high_risk_mae_ratio	۰.۳۵ / ۰.۶۰ / ۰.۸۵	مرزهای طبقه‌بندی «درد مسیر» معامله.
extreme/high_volatility_percentile	۰.۸۰ / ۰.۹۵	صدک نوسان برای تشدید رده ریسک.
discard_very_high_risk	true	معاملات با ریسک خیلی بالا به «معامله نکن» تبدیل می‌شوند.

۷ MLSettings – یادگیری ماشین

کلید	پیش فرض	معنی
model_dir	artifacts/models	محل ذخیره فایل‌های مدل.
booster	lightgbm	الگوریتم: LightGBM (یا XGBoost).
n_estimators	۴۰۰	تعداد درخت‌ها.
learning_rate	۰.۰۵	سرعت یادگیری (کم = محتاط).
max_depth / num_leaves	۶۳ / ۶	عمق و پیچیدگی درخت‌ها.
subsample / colsample	۰.۸۵	هر درخت ۸۵٪ ردیف‌ها و ۸۵٪ ستون‌ها را می‌بیند (ضد حفظ‌کردن).
validation_fraction	۰.۲	۲۰٪ انتهایی داده برای آزمون کنار گذاشته می‌شود.
purge_bars	۶۰	۶۰ کندل بین آموزش و آزمون حذف می‌شود (خیلی مهم — بخش ۱۳).
early_stopping_rounds	۵۰	اگر ۵۰ دور پیاپی بهتر نشد، آموزش را قطع کن.

۸ DecisionSettings – آستانه‌های موتور تصمیم

کلید	پیش فرض	معنی
min_direction_confidence	۰.۷۰	مدل جهت باید حداقل ۷۰٪ مطمئن باشد.
min_direction_margin	۰.۱۵	فاصله لانگ و شورت حداقل ۱۵ واحد درصد.
max_no_trade_probability	۰.۳۵	احتمال «معامله نکن» نباید از ۳۵٪ بیشتر شود.
min_entry_probability	۰.۵۵	مدل زمان‌بندی ورود باید حداقل ۵۵٪ بگوید «الان».
min/max_leverage	۱ / ۱۰	بازه مجاز اهرم.
min/max_capital_allocation_pct	۰.۲۰ / ۰.۰۱	بین ۱٪ و ۲۰٪ سرمایه در هر معامله.
min_reward_risk_ratio	۱۳	نسبت سود به ضرر حداقل ۱۳ باشد.
max_volatility_percentile	۰.۹۷	در نوسان صدک ۹۷ به بالا، معامله ممنوع.
accepted_risk_tiers	LOW, MEDIUM, HIGH	رده VERY_HIGH پذیرفته نمی‌شود.
max_concurrent_positions	۵	حداکثر ۵ معامله باز همزمان.
max_positions_per_symbol	۱	روی هر ارز فقط یک معامله.

۹ RiskSettings – نگاهان ریسک

کلید	پیش فرض	معنی
starting_equity	۱۰۰۰	سرمایه اولیه فرضی.

daily_drawdown_yellow / red	۰.۰۳ / ۰.۰۵	افت روزانه ۳٪ = زرد، ۵٪ = قرمز (توقف کامل).
total_drawdown_red_pct	۰.۲۰	افت ۲۰٪ از اوج = توقف کامل.
consecutive_losses_yellow / red	۵ / ۳	۳ باخت پیاپی = زرد، ۵ باخت = قرمز.
api_errors_yellow / red	۶ / ۳	۶ خطای صرافی در ۵ دقیقه = قرمز.
yellow_size_multiplier	۰.۴	در حالت زرد، حجم معامله ۴۰٪ می‌شود.
max_daily_trades	۴۰	حداکثر ۴۰ معامله در روز.
require_manual_reset	true	حالت قرمز فقط با دست آدم باز می‌شود.

۱۰. ExecutionSettings – اجرا

کلید	پیش فرض	معنی
margin_mode	isolated	حالت مارجین ایزوله: ضرر هر معامله فقط به ودیعه خودش محدود است.
order_type	market	سفارش بازار (اجرای فوری).
taker_fee / maker_fee	۰.۰۰۰۵ / ۰.۰۰۰۲	۰.۰۵٪ و ۰.۰۲٪.
slippage_bps	۵	۰.۰۵٪ لغزش قیمت در شبیه‌سازی.
position_monitor_interval_seconds	۵	هر ۵ ثانیه پوزیشن‌ها چک می‌شوند.
reconcile_interval_seconds	۶۰	هر ۶۰ ثانیه با صرافی تطبیق داده می‌شود.
funding_interval_hours	۸	فاندینگ هر ۸ ساعت.
maintenance_margin_rate	۰.۰۰۵	نرخ مارجین نگهداری برای محاسبه قیمت لیکوئید.
paper_starting_balance	۱۰۰۰	موجودی مجازی حالت کاغذی.

۱۱. WebSettings و ۱۲. DatabaseSettings

کلید	پیش فرض	معنی
web.host / port	8000 / 0.0.0.0	پنل روی همهٔ رابط‌ها، پورت ۸۰۰۰.
web.api_token	خالی	اگر پر شود، دستورهای حساس رمز می‌خواهند.
db.path	data/quant.db	یک فایل SQLite. بدون سرور دیتابیس.
db.journal_mode	WAL	حالت WAL: پنل می‌تواند بخواند در حالی که ربات می‌نویسد.
db.busy_timeout_ms	۱۰۰۰۰	اگر فایل قفل بود، ۱۰ ثانیه صبر کن.

۳-۳. اعتبارسنجی‌های خودکار تنظیمات

فایل تنظیمات خودش را هم چک می‌کند و اگر ترکیب بی‌معنی باشد، برنامه اصلاً بالا نمی‌آید:

- min_leverage نمی‌تواند از max_leverage بزرگ‌تر باشد.
- آستانهٔ زرد افت سرمایه باید کمتر از آستانهٔ قرمز باشد.
- سطح لاگ باید یکی از پنج مقدار مجاز باشد.
- هنگام ساخت تنظیمات، پوشه‌های logs/، artifacts/models و پوشهٔ پایگاه داده بلافاصله ساخته می‌شوند تا بعداً هیچ عملیات فایلی شکست نخورد.

تابع `get_settings()` با `@lru_cache` نگه داشته شده، یعنی تنظیمات در کل عمر برنامه فقط یک‌بار خوانده می‌شود و همه از همان یک نسخه استفاده می‌کنند.

۴. ابزارهای پایه — پوشه core/

👤 **به زبان بچه‌گانه:** اینجا جعبه‌ابزار مشترک است: یک ساعت، یک دفترچه یادداشت، و یک لیست از انواع «داد زدن» وقتی چیزی خراب می‌شود.

۴-۱. exceptions.py — انواع خطا

به‌جای اینکه همه‌جا فقط بگوییم «خطا شد»، ۱۲ نوع خطای مشخص تعریف شده تا بشود دقیق واکنش نشان داد. همه از یک پدر مشترک به نام QuantSystemError ارث می‌برند که می‌تواند «زمینه» (context) هم حمل کند — مثلاً کدام ارز، چند بار تلاش شد.

خطا	کی برناب می‌شود
ConfigurationError	تنظیمات با خودش ناسازگار است.
DataFetchError	صرافی بعد از همه تلاش‌ها داده نداد.
DataIntegrityError	پلیس کیفیت نتوانست داده را تعمیر کند.
InsufficientDataError	تعداد ردیف‌ها برای محاسبه کافی نیست.
FeatureEngineeringError	ساخت یک ویژگی به‌طور جبران‌ناپذیر شکست خورد.
LabelingError	برچسب‌زن ورودی ناسازگار گرفت.
ModelNotLoadedError	پیش‌بینی قبل از بارگذاری مدل.
ModelTrainingError	مدل قابل آموزش نبود (مثلاً فقط یک کلاس در داده).
DecisionEngineError	ورودی بدشکل به موتور تصمیم.
ExecutionError	سفارش ثبت/لغو نشد.
KillSwitchEngaged	نگهبان ریسک قرمز است و ریسک جدید ممنوع.
DatabaseError	عملیات پایگاه داده شکست خورد.

۴-۲. logger.py — دفترچه یادداشت

سه مقصد برای هر پیام لاگ:

۱. **ترمینال** — چیزی که روی صفحه می‌بینی.
 ۲. **فایل چرخشی** — logs/quant_system.log، هر فایل تا ۲۵ مگابایت، ۵ فایل پشتیبان نگه داشته می‌شود.
 ۳. **حافظه حلقوی** — کلاس RingBufferHandler آخرین ۸۰۰ خط را در حافظه نگه می‌دارد. پنل وب از همین می‌خواند، پس برای دیدن لاگ زنده نیازی به خواندن فایل از دیسک نیست.
- همچنین کتابخانه‌های پرحرف (ccxt, urllib3, asyncio, apscheduler, aioredis) به سطح WARNING خفه می‌شوند تا لاگ ما را پر نکنند. یک پرچم CONFIGURED_ تضمین می‌کند تنظیم لاگ فقط یک‌بار انجام شود.

۴-۳. utils.py — توابع کوچک اما حیاتی

الف) کار با زمان

کل سیستم با میلی‌ثانیه از مبدأ یونیکس، به وقت UTC کار می‌کند. هیچ منطقه زمانی محلی در کار نیست.

تابع	کار
utc_now() / utc_now_ms()	زمان فعلی UTC.

ms_to_datetime()	میلی‌ثانیه ← تاریخ خوانا.
floor_to_timeframe()	گرد کردن زمان روی شبکه ۵ دقیقه‌ای. چون کندل‌ها همیشه روی مضرب دقیق ۳۰۰'۰۰۰ باز می‌شوند، این فقط یک عملیات باقیمانده ساده است.
last_closed_candle_open_ms()	زمان باز شدن آخرین کندلی که واقعاً بسته شده است.
expected_timestamps()	لیست همه زمان‌هایی که باید بین دو نقطه وجود داشته باشند — اساس تشخیص «کندل گمشده».

ب) کار با عدد

- `safe_float()` — تبدیل هر چیزی به عدد؛ اگر نشد یا نتیجه NaN/بی‌نهایت بود، مقدار پیش‌فرض برمی‌گرداند. این یک سپر مهم در برابر پاسخ‌های عجیب صرافی است.
- `clamp()` — محدود کردن عدد در یک بازه.
- `percentile_rank()` — چند درصد داده‌ها کمتر یا مساوی این مقدارند.
- `chunked()` — تکه‌تکه کردن لیست‌های بزرگ.

ج) تلاش مجدد هوشمند — قلب پایداری شبکه

تابع `backoff_delay(attempt, base, max, jitter)` تأخیر را **نمایی** زیاد می‌کند: ۱، ۲، ۴، ۸، ۱۶ ثانیه... با سقف، و با $\pm 25\%$ نویز تصادفی.

چرا این‌طوری؟ چرا نویز تصادفی؟ فرض کن ۳۰ ارز هم‌زمان خطا خوردند. اگر همه دقیقاً ۲ ثانیه بعد دوباره تلاش کنند، همان لحظه یک موج ۳۰ تایی به صرافی می‌خورد و باز هم خطا می‌گیرند. نویز تصادفی این موج را پخش می‌کند.

تابع `async_retry()` این را عملی می‌کند و سه دسته خطا را از هم جدا می‌کند:

- `retry_on` — خطاهای گذرا (قطعی شبکه، تایم‌اوت، محدودیت نرخ): دوباره تلاش کن.
- `give_up_on` — خطاهای دائمی (نماد اشتباه، کلید API نامعتبر): بلافاصله تسلیم شو.
- `asyncio.CancelledError` — همیشه بدون دست‌کاری عبور داده می‌شود (این خطای «لغو» است، نه خطای واقعی).

نکته: تفکیک خطاهای گذرا از دائمی خیلی مهم است: تلاش مجدد روی یک «کلید API اشتباه» فقط سهمیه درخواست تو را می‌سوزاند و هیچ‌وقت موفق نمی‌شود.

۵. لحظه روشن شدن سیستم — main.py

وقتی دستور زیر را می‌زنی:

```
python main.py
```

این اتفاق‌ها به ترتیب می‌افتد:

۵-۱. گام‌های راه‌اندازی (تابع startup())

۱ خواندن آرگومان‌های خط فرمان — تابع `_parse_args()` تعیین می‌کند کدام دستور اجرا شود: `run` (پیش‌فرض)، `universe`, `bootstrap`, `train`, `backtest` یا `cycle`.

۲ ساخت تنظیمات و راه‌اندازی لاگ — `get_settings()` و `configure_logging()`.

۳ ساخت شیء `TradingSystem` — در سازنده آن، همه اجزای شش ماژول یک‌جا ساخته و به هم وصل می‌شوند: پایگاه داده، دریافت‌کننده داده، اعتبارسنج، خط لوله، مدیر ارزها، سرویس ویژگی، پردازشگر دیتاست، زیرسیستم ML، موتور تصمیم، نگهدارنده ریسک، موتور اجرا و موتور ثبت وقایع.

۴ آماده‌سازی پایگاه داده — `database.initialize()`: فایل `data/quant.db` ساخته می‌شود، حالت WAL فعال می‌شود و ۶ جدول اگر نبودند ساخته می‌شوند.

۵ روشن کردن موتور ثبت وقایع — یک تسک پس‌زمینه که هر ۲ ثانیه صف را خالی می‌کند.

۶ بازبازی وضعیت نگهدارنده ریسک — `risk_guard.load()`. اگر در اجرای قبلی سیستم «قرمز» شده بود، همان الان دوباره قرمز می‌شود. این خیلی مهم است: ری‌استارت کردن برنامه راه فرار از توقف اضطراری نیست.

۷ بارگذاری فهرست بازارهای بایننس — `fetcher.load_markets()`. فقط بازارهای «خطی» (USDT-M) بارگذاری می‌شوند، نه اسپات و نه معکوس.

۸ بارگذاری چهار مدل از دیسک — اگر فایلی نبود، پیام می‌دهد که در مرحله راه‌اندازی آموزش داده خواهند شد.

۹ خواندن انتخاب ارزهای ذخیره‌شده از پایگاه داده.

۱۰ ثبت تابع «تخلیه اضطراری» در نگهدارنده ریسک، و روشن کردن موتور اجرا.

چرا این‌طوری؟ چرا مرحله راه‌اندازی این‌قدر سریع و «تحميل‌کننده خطای شبکه» طراحی شده؟ چون پنل وب باید بالا بیاید حتی وقتی صرافی در دسترس نیست — تا اپراتور بتواند ببیند مشکل چیست. کارهای سنگین (دانلود تاریخچه و آموزش) بعداً در پس‌زمینه انجام می‌شوند.

۵-۲. زمان‌بند ۵ دقیقه‌ای

تابع `_schedule()` یک کار زمان‌بندی‌شده ثبت می‌کند:

```
CronTrigger(minute="0,5,10,...,55", second=10, timezone="UTC")
```

پارامتر	معنی
<code>max_instances=1</code>	هرگز دو چرخه همزمان اجرا نشوند.
<code>coalesce=True</code>	اگر چند اجرا عقب افتاد، فقط یکی اجرا شود نه همه.
<code>misfire_grace_time=90</code>	تا ۹۰ ثانیه تأخیر قابل قبول است؛ بیشتر از آن، آن نوبت رد می‌شود.

علاوه بر این، خود تابع `trading_cycle()` یک قفل دارد: اگر چرخه قبلی هنوز تمام نشده، چرخه جدید **رد می‌شود** (نه اینکه در صف بماند).

۵-۳. اجرای همزمان پنل و زمان‌بند

تابع `run_forever()` دو کار را همزمان روی یک حلقه رویداد اجرا می‌کند: سرور `uvicorn` (پنل وب) و زمان‌بند. با `asyncio.wait(..., FIRST_COMPLETED)` منتظر می‌ماند تا یکی از این دو تمام شود یا سیگنال خاموشی برسد.

۵-۴. خاموشی تمیز

سیگنال‌های `SIGINT` (Ctrl+C) و `SIGTERM` گرفته می‌شوند و خاموشی به ترتیب معکوس انجام می‌شود:

۱. پرچم معامله خاموش می‌شود.
۲. تسک راه‌اندازی (اگر در حال اجراست) لغو می‌شود.
۳. زمان‌بند متوقف می‌شود.
۴. موتور اجرا متوقف می‌شود (پوزیشن‌های باز خودکار بسته نمی‌شوند).
۵. صف ثبت وقایع کاملاً روی دیسک خالی می‌شود.
۶. سرویس ویژگی‌سازی خاموش می‌شود.
۷. اتصال به صرافی بسته می‌شود.
۸. اتصال به پایگاه داده بسته می‌شود.

۵-۵. خط لوله خودکار راه‌اندازی (`run_setup`)

بعد از بالا آمدن، یک تسک پس‌زمینه شروع می‌شود که سه حالت دارد:

```
اگر هیچ ارزی انتخاب نشده باشد
|
| (منتظر می‌ماند؛ تو باید در پنل ارز تیک بزنی) Awaiting_universe فاز ++>
|
اگر ارز انتخاب شده باشد
|
| داندلود تاریخچه _setup_collect(): مرحله ۱ ++>
|
| آموزش ۴ مدل _setup_train(): مرحله ۲ ++>
|
| (ولی هنوز معامله نمی‌کند - آماده) Ready فاز ++>
```

جزئیات `_setup_collect()`

- فاز به `COLLECTING_DATA` می‌رود و نوار پیشرفت شروع می‌شود.

- `pipeline.bootstrap_history()` صدا زده می‌شود با یک تابع `callback` که هر بار یک ارز تمام شد، نوار پیشرفت را جلو می‌برد.
- بعد از پایان، تعداد کندل‌های ذخیره‌شده هر ارز شمرده می‌شود.
- ارزی که کمتر از **۵۰۰ کندل** دارد «غیرقابل استفاده» علامت می‌خورد.
- اگر هیچ ارزی به ۵۰۰ کندل نرسید، خطا داده می‌شود: احتمالاً اتصال به صرافی مشکل دارد.

جزئیات `_setup_train()`

آموزش **رد می‌شود** اگر هر سه شرط برقرار باشد:

1. هر چهار مدل روی دیسک موجود و بارگذاری شده باشند،
2. لیست ارزهای انتخابی با لیستی که مدل‌ها روی آن آموزش دیده‌اند یکسان باشد،
3. کاربر آموزش اجباری نخواست باشد.

? چرا این‌طوری؟ چرا تغییر لیست ارزها باعث آموزش مجدد اجباری می‌شود؟ چون مدل روی یک «مقطع عرضی» خاص از بازار برازش شده. اگر ارزها عوض شوند، آن مدل دیگر درباره این بازار حرف نمی‌زند. لیست ارزهای آموزش در کلید `trained_universe` پایگاه داده ذخیره می‌شود.

در غیر این صورت: ساخت دیتاست ← آموزش ۴ مدل ← ذخیره نتایج و آمار.

۵-۶. مسلح کردن معامله `(start_trading)`

سیستم قبل از مسلح شدن ۵ شرط را چک می‌کند و اگر هرکدام رد شود، خطا می‌دهد:

#	شرط
۱	حالت باید «paper» یا «live» باشد.
۲	فاز فعلی باید اجازه مسلح شدن بدهد (یعنی راه‌اندازی تمام شده باشد).
۳	نگه‌بان ریسک نباید قرمز باشد.
۴	برای حالت واقعی: هر چهار مدل باید مدل واقعی آموزش‌دیده باشند، نه قاعده جایگزین.
۵	حداقل یک ارز باید انتخاب شده باشد.

اگر حالت عوض شود (کاغذی ← واقعی)، تابع `_switch_engine()` ابتدا **همه پوزیشن‌های موتور قبلی را می‌بندد**، بعد موتور جدید را می‌سازد.

? چرا این‌طوری؟ چرا؟ چون یک پوزیشن کاغذی هیچ معادلی روی صرافی ندارد و برعکس. اگر حین تعویض موتور پوزیشنی باقی بماند، تا ابد بدون مدیریت رها می‌شود.

۶. مرحله ۱: انتخاب ارزها — universe.py

👤 **به زبان بچه‌گانه:** فرض کن می‌خواهی تیم فوتبال بسازی. ۴۰۰ بازیکن جلوی تو هستند. چهار آزمون می‌گیری: قد، سرعت، سابقه، و اینکه لباس تیم اندازه‌اش بشود. هر بازیکنی که در یکی رد شد، کنار می‌رود — و روی برگه‌اش دقیقاً می‌نویسی در کدام آزمون رد شد. این فایل دقیقاً همین کار را با ارزها می‌کند.

۶-۱. کشف ارزها (discover())

فقط با دو درخواست به بایننس:

1. load_markets() — فهرست کامل بازارها با مشخصات فنی هرکدام.

2. fetch_tickers() — قیمت لحظه‌ای، بهترین خرید/فروش و حجم ۲۴ ساعته برای همه در یک درخواست.

سپس فیلتر اولیه: یک بازار فقط وقتی «قرارداد دائمی USDT-M» شمرده می‌شود که

```
swap == True and linear == True and active == True
and settle == "USDT" and quote == "USDT" and not expiry
```

نتیجه ۱۵ دقیقه در حافظه کش می‌شود (discovery_cache_seconds = 900) چون مشخصات بازار دقیقه‌به‌دقیقه عوض نمی‌شود.

۶-۲. چهار غربال

غربال ۱: نقدشوندگی

حجم معاملات ۲۴ ساعته باید ≤ 50 میلیون USDT باشد.

چرا این‌طوری؟ در یک بازار کم‌عمق، خود سفارش تو قیمت را جابه‌جا می‌کند. با اهرم ۱۰، تو عملاً به خودت ضرر می‌زنی قبل از اینکه تحلیل فرصت درست‌بودن داشته باشد.

غربال ۲: اسپرد (فاصله خرید/فروش)

فرمول: $6 \leq (ask - bid) / mid \times 10000$ پایه.

چرا این‌طوری؟ در یک استراتژی ۵ دقیقه‌ای، اسپرد در هر رفت‌وبرگشت پرداخت می‌شود. اسپرد ۱۵ پایه، یک معامله ۲:۱ را قبل از شروع می‌بلعد.

نکته طریف: اگر اسپرد صفر گزارش شود، یعنی تیکر اصلاً دفتر سفارش نداشته. کد این را قبول می‌کند تا نبود داده به‌جای داده بد تفسیر نشود.

غربال ۳: قدمت

بایننس تاریخ لیست‌شدن هر قرارداد را در فیلد onboardDate می‌دهد، پس این غربال هیچ درخواست اضافه‌ای لازم ندارد. حداقل ۹۰ روز.

چرا این‌طوری؟ زیر ۹۰ روز، اصلاً به‌اندازه کافی کندل ۵ دقیقه‌ای وجود ندارد که مدل چیزی یاد بگیرد.

غربال ۴: تناسب با سرمایه کوچک — مهم‌ترین و کم‌بای‌ترین غربال

ابتدا «کوچک‌ترین پوزیشن ممکن» حساب می‌شود:

```
smallest = reference_equity * min_capital_allocation_pct * min_leverage
           = 1000 * 0.01 * 1 = 10 USDT
```

شرط	معنی
قابل خرید بودن	حداقل ارزش سفارش صرافی (بزرگ‌تر حداقل مقدار $\times$ قیمت، و حداقل ارزش) نباید از ۱۰ USDT بیشتر باشد.
دانه‌بندی	هزینه یک پله مقدار (amount_step $\times$ price) نباید از ۲۵٪ آن ۱۰ USDT (یعنی ۲٫۵ USDT) بیشتر باشد.

👉 **به زبان بچه‌گانه:** مثال بیت‌کوین: پله مقدار ۰٫۰۰۱ بیت‌کوین است. اگر بیت‌کوین ۹۵'۰۰۰ دلار باشد، یک پله ۹۵ دلار می‌شود. تو می‌خواستی ۱۰ دلار معامله کنی، ولی کوچک‌ترین قدمی که می‌توانی برداری ۹۵ دلار است! مثل اینکه بخواهی ۱۰۰ گرم شکر بخری ولی کوچک‌ترین بسته ۱ کیلو باشد. پس بیت‌کوین — با تمام نقدشوندگی عالی‌اش — برای یک حساب کوچک **غیرقابل استفاده** است و رد می‌شود.

نکتهٔ ظریف در تابع `_amount_step()`: کتابخانهٔ `ccxt` گاهی `precision.amount` را به صورت «اندازهٔ پله» (۰٫۰۰۱) و گاهی به صورت «تعداد رقم اعشار» (۳) می‌دهد. کد هر دو حالت را تشخیص می‌دهد: عدد صحیح ≤ 1 یعنی تعداد رقم اعشار و به 10^{-n} تبدیل می‌شود.

۶-۳. امتیازدهی و رتبه‌بندی (`_apply_scores`)

چهار معیار نرمال‌سازی و با وزن ترکیب می‌شوند:

معیار	وزن	نحوهٔ محاسبه
نقدشوندگی	۰٫۴۵	$\log_{10}(\text{volume})$ سپس نرمال‌سازی min-max
اسپرد	۰٫۲۵	نرمال‌سازی معکوس (کمتر = بهتر)
مقرون‌به‌صرفه بودن	۰٫۲۰	$\log_{10}(\text{granularity})$ معکوس
قدمت	۰٫۱۰	روزهای لیست‌شدن، سقف ۱۰۹۵ روز (۳ سال)

❓ **چرا این‌طوری؟** چرا لگاریتم؟ چون حجم ۲۴ ساعته چهار مرتبهٔ بزرگی تفاوت دارد (از ۵۰ میلیون تا ۵۰ میلیارد). بدون لگاریتم، فقط یکی دو ارز برتر امتیاز می‌گرفتند و بقیه صفر می‌شدند.

در پایان، لیست ابتدا بر اساس «واجد شرایط بودن» و سپس بر اساس امتیاز مرتب می‌شود.

۶-۴. ذخیرهٔ انتخاب (`save_selection`)

- هر نماد در برابر لیست بازارهای کشف‌شده اعتبارسنجی می‌شود. نماد اشتباه یا قرارداد حذف‌شده **همین‌جا** رد می‌شود، نه در چرخهٔ بعدی.
- اگر کاربر عمداً یک ارز ردشده را تیک بزند، سیستم *احترام می‌گذارد* ولی یک هشدار در لاگ می‌نویسد.
- اگر لیست خالی باشد، خطا داده می‌شود.
- مقایسه با انتخاب قبلی برای تعیین `changed` — که آموزش مجدد را اجباری می‌کند.
- ذخیره در کلید `universe_selection` پایگاه داده همراه با زمان و نام اپراتور.

۶-۵. پیشنهاد خودکار (`suggest`)

فقط ارزهای **واجد شرایط** را بر اساس امتیاز مرتب می‌کند و N تای اول را برمی‌گرداند. اگر تعداد کافی نبود، لیست کوتاه‌تر برمی‌گردد — هرگز با ارزهای ردشده پر نمی‌شود.

۷. مرحله ۲: جمع‌آوری داده از بایننس — `fetcher.py`

👉 **به زبان بچه‌گانه:** این فایل «مأمور خرید» است. می‌رود به مغازه (بایننس)، جنس می‌خرد، و اگر مغازه شلوغ بود یا در بسته بود، چند بار دیگر با فاصله بیشتر برمی‌گردد. ولی اگر مغازه بگوید «این جنس اصلاً وجود ندارد»، بی‌خودی برنمی‌گردد.

۷-۱. آماده‌سازی اتصال

یک شیء `ccxt.async_support.binance` ساخته می‌شود با کلیدها، تایم‌اوت، محدودکننده نرخ داخلی، و `defaultType="future"`.

یک ترفند مهم: محدود کردن به بازارهای خطی

```
exchange.options["fetchMarkets"] = ["linear"] # یا {"types": ["linear"]}
```

❓ **چرا این‌طوری؟** `CCXT` به‌صورت پیش‌فرض بازارهای اسپات، خطی و معکوس را بارگذاری می‌کند — چند مگابایت داده و مقدار زیادی از سهمیه درخواست که این سیستم اصلاً استفاده نمی‌کند. بدتر: در حالت آزمایشی، بخش اسپات از یک دامنه کاملاً جدا (`testnet.binance.vision`) سرو می‌شود، و یک ربات فیوچرز به دلیلی که هیچ ربطی به فیوچرز ندارد بالا نمی‌آید. کد شکل قدیمی (لیست) و جدید (دیکشنری) این تنظیم را هر دو پشتیبانی می‌کند.

۷-۲. لایه محافظ همه درخواست‌ها (`_call`)

هیچ درخواستی مستقیم ارسال نمی‌شود؛ همه از این تابع عبور می‌کنند که:

۱. یک `Semaphore` می‌گیرد (حداکثر ۸ درخواست همزمان).
۲. با `async_retry` تا ۵ بار با تأخیر نمایی تلاش می‌کند.
۳. خطاها را دسته‌بندی می‌کند.
۴. شمارنده `consecutive_errors` را به‌روز می‌کند (نگهبان ریسک از آن استفاده می‌کند).

دسته	شامل	واکنش
گذرا	<code>NetworkError</code> , <code>RequestTimeout</code> , <code>ExchangeNotAvailable</code> , <code>DDoSProtection</code> , <code>RateLimitExceeded</code> , <code>OnMaintenance</code> , <code>TimeoutError</code>	تلاش مجدد
دائمی	<code>AuthenticationError</code> , <code>PermissionDenied</code> , <code>BadSymbol</code> , <code>ArgumentsRequired</code>	تسلیم فوری
سایر خطای صرافی	<code>ExchangeError</code>	تسلیم با گزارش

۷-۳. دریافت کندل‌ها

الف) `fetch_ohlcv()` — یک صفحه

- حداکثر ۱۵۰۰ کندل در یک درخواست.
- هر ردیف خام به کلاس `OHLCVCandle` تبدیل می‌شود. اگر ردیفی ساختار شکسته داشت، حذف می‌شود (نه اینکه کل درخواست شکست بخورد) — و پلیس کیفیت بعداً حفره ایجادشده را می‌بیند و تعمیر می‌کند.
- کندل هنوز بسته نشده حذف می‌شود: اگر `timestamp + 300000 > now` باشد، آن کندل نیمه‌تمام است.
- نتیجه بر اساس زمان مرتب می‌شود.

ب) `fetchohlcv_range()` — یک بازه طولانی با صفحه‌بندی

برای گرفتن ۲ سال تاریخچه (۲۱۰'۲۴۰ کندل) باید حدود ۱۴۰ صفحه گرفت. حلقه از `start_ms` شروع می‌کند و هر بار مکان‌نما را به آخرین `m` کندل + 5 می‌برد. سه محافظ در برابر حلقه بی‌نهایت:

- صفحه خالی ← توقف.
 - سقف تعداد صفحات (`max_pages`) محاسبه‌شده از طول بازه.
 - اگر مکان‌نما جلو نرفت و هیچ کندل جدیدی نیامد ← توقف.
- کندل‌ها در یک دیکشنری با کلید زمان جمع می‌شوند، پس تکراری غیرممکن است.

۷-۴. دفتر سفارش (`fetch_order_book`)

عمق ۲۰ سطح گرفته می‌شود و بلافاصله به آمار خلاصه تبدیل می‌شود (کل دفتر ذخیره نمی‌شود): بهترین خرید، بهترین فروش، حجم ۱۰ سطح هر طرف. از آن‌ها این‌ها محاسبه می‌شود:

کمیت	فرمول
قیمت میانی	$(bid + ask) / 2$
اسپرد	$ask - bid$
اسپرد نسبی	$(spread / mid) \times 10000$ پایه
عدم توازن	$(bid_vol - ask_vol) / (bid_vol + ask_vol)$ در بازه ۱- تا ۱+
ریزقیمت	$(bid \times ask_vol + ask \times bid_vol) / (bid_vol + ask_vol)$

اگر دفتر خالی یا «مقاطع» بود (خرید $\leq$ فروش، که نشانه اسنپ‌شات کهنه است)، تابع `None` برمی‌گرداند — نه یک عدد اشتباه.

۷-۵. متریک‌های قرارداد آتی (`fetch_futures_metrics`)

شش درخواست موازی با `asyncio.gather`:

داده	در صورت شکست
نرخ فاندینگ + قیمت مارک/شاخص	۰٫۰
بهره باز (Open Interest)	۰٫۰
نسبت لانگ/شورت کل حساب‌ها	۱٫۰ (کاملاً متعادل)
نسبت لانگ/شورت معامله‌گران برتر	۱٫۰
نسبت خرید/فروش تیکرها	۱٫۰
حجم لیکوئیدیشن	(۰٫۰ , ۰٫۰)

؟ چرا این‌طوری؟ هر زیر-درخواست جداگانه محافظت شده است. نقاط انتهایی `futures/data` بایننس معروف به بی‌ثباتی‌اند، و ارزش متوقف کردن معامله را ندارند. پس شکست یکی، فقط همان یک فیلد را به مقدار خنثی تبدیل می‌کند، نه اینکه کل چرخه را از بین ببرد.

یک محافظ اضافه: اگر نرخ فاندینگ بزرگ‌تر از ۵٪ گزارش شد، صفر می‌شود — چون قالب `Pydantic` چنین مقداری را رد می‌کند و کل بسته را از بین می‌برد.

۷-۶. قالب‌های سختگیرانه داده (`models.py`)

اینجا اولین خط دفاعی است. یک کندل در همان لحظه ساخته‌شدن این قوانین را چک می‌کند:

```
high >= low
high >= max(open, close)
low <= min(open, close)
all prices > 0
volume >= 0
timestamp >= 1262304000000 # 2010-01-01 : کف سلامت عقل
```

🚩 **نکته:** تنظیم frozen=True یعنی کندل پس از ساخت **غیرقابل تغییر** است. هیچ‌جای کد نمی‌تواند مخفیانه قیمتی را عوض کند. و extra="forbid" یعنی هیچ فیلد ناشناخته‌ای پذیرفته نمی‌شود.

کلاس MarketDataBundle خروجی نهایی مازول A برای هر ارز است: کندل‌ها + دفتر سفارش + متریک‌های آتی + گزارش کیفیت + زمان دریافت. متد () to_dataframe() آن را به جدول pandas با شاخص زمانی UTC تبدیل می‌کند — دقیقاً چیزی که مازول B انتظار دارد.

۸. مرحله ۳: پلیس کیفیت داده — qc_validator.py

👉 **به زبان بچه‌گانه:** این آقا بازرس است. جعبه میوه که رسید، دانه‌دانه نگاه می‌کند: میوه گندیده هست؟ جای خالی هست؟ یک میوه دو بار شمرده شده؟ میوه‌ای از فردا آمده؟! اگر مشکلی دید، فقط همان قسمت خراب را دوباره سفارش می‌دهد. اگر بعد از ۴ بار باز هم درست نشد، کل جعبه را برای این نوبت کنار می‌گذارد — **ولی هرگز میوه پلاستیکی جایش نمی‌گذارد.**

۸-۱. چهار خانواده بررسی

خانواده ۱: یکپارچگی زمانی

بررسی	شدت	توضیح
هم‌ترازی با شبکه	بحرانی	هر زمان باید مضرب دقیق ۳۰۰'۰۰۰ میلی‌ثانیه باشد.
تکراری	بحرانی	دو کندل با یک زمان باز شدن.
نامرتب	هشدار	کندل‌ها به‌هم‌ریخته آمده‌اند. خودکار مرتب می‌شوند و فقط یادداشت می‌شود.
زمان آینده	بحرانی	کندلی که تاریخش از حالا جلوتر است.
کندل گمشده	بحرانی	محاسبه شبکه کامل بین اولین و آخرین کندل و پیدا کردن نقاط غایب. لیست دقیق زمان‌های گمشده برگردانده می‌شود.
کهنگی	بحرانی / اطلاع	اگر آخرین کندل بیش از ۳ کندل عقب باشد ← بحرانی؛ ۱ تا ۳ کندل ← فقط اطلاع.

خانواده ۲: منطق قیمت

همان قوانین $high \geq \max(o,c) \geq \min(o,c) \geq low$ و مثبت بودن و متناهی بودن (نه NaN نه بی‌نهایت). این‌ها در قالب Pydantic هم چک می‌شوند، پس اگر اینجا نقض شوند یعنی داده از مسیر دیگری وارد شده — باز هم بحرانی محسوب می‌شود.

خانواده ۳: منطق حجم

- حجم منفی ← بحرانی.
- بیش از ۲۵٪ کندل‌ها حجم صفر ← بحرانی (یعنی فید مرده است، نه بازار آرام).
- حجمی بیش از ۵۰ برابر میانه حجم‌های مثبت ← فقط هشدار (چون گاهی واقعاً یک انفجار حجمی واقعی رخ می‌دهد).

خانواده ۴: ناهنجاری‌های آماری — قسمت هوشمندانه

ابتدا بازدهی لگاریتمی محاسبه می‌شود: $\text{diff}(\log(\text{close}))$. سپس دو آزمون:

1. **سقف مطلق:** هیچ کندل ۵ دقیقه‌ای نباید بیش از ۳۵٪ حرکت کند ← بحرانی.

2. **z-score مقاوم بر پایه MAD:**

```
median = median(log_returns)
MAD    = median( |log_returns - median| )
sigma  = 1.4826 * MAD
z       = |log_returns - median| / sigma
if z > 14 -> WARNING
```

❓ **چرا این طوری؟ چرا MAD و نه انحراف معیار معمولی؟** این نکته کلیدی است. فرض کن ۱۰۰ کندل داری که همه ۰.۱٪ حرکت کرده‌اند، و یک کندل خراب که ۴۰٪ پریده. اگر انحراف معیار حساب کنی، خود همان کندل خراب انحراف معیار را آن‌قدر بزرگ می‌کند که دیگر «غیرعادی» به‌نظر نمی‌رسد و از دست تو فرار می‌کند! میانه انحراف مطلق (MAD) نقطه شکست ۵۰٪ دارد؛ تا وقتی کمتر از نصف

داده‌ها خراب نشده‌اند، MAD به بدنهٔ سالم داده چسبیده می‌ماند. ضرب ۱,۴۸۲۶ فقط برای این است که MAD را به مقیاس سیگمای توزیع نرمال تبدیل کند.

جزئیات ظریف: کندلهایی که در آزمون سقف مطلق گیر افتاده‌اند از آزمون MAD حذف می‌شوند ($\text{soft_mask} \&= \sim \text{hard_mask}$) تا یک کندل دوبار گزارش نشود.

۸-۲. تعمیر خودکار (validate_and_heal)

اعتبارسنجی

```
|
| تمام، برگرد --> بله --> ? قبول شد ---+
|
| نه
|
| DataIntegrityError پرتاب --> نه --> ? آیا قابل تعمیر است ---+
|
| بله
|
| (۲، ۴، ۸، ۱۶ ثانیه) صبر با تأخیر نمایی ---+
| _heal_window() محاسبه پنجره آسیب‌دیده ---+
| فقط همان پنجره را دوباره از صرافی بگیر ---+
| کندلهای قدیمی داخل پنجره دور ریخته میشوند :ادغام ---+
| (حداکثر ۴ بار) دوباره اعتبارسنجی کن ---+
|
| ارز این چرخه رد میشود --> DataIntegrityError --> اگر باز هم خراب بود ---+
```

محاسبه پنجره تعمیر

همهٔ زمان‌های مشکوک (کندلهای گمشده + زمان‌های پیوست‌شده به مشکلات بحرانی) جمع می‌شوند، کمینه و بیشینه‌شان گرفته می‌شود و یک کندل از هر طرف حاشیه اضافه می‌شود تا کندلهای مرزی هم تازه شوند.

ادغام (_merge_blocks)

همهٔ کندلهای قدیمی داخل پنجره آسیب‌دیده دور ریخته می‌شوند و نسخهٔ تازهٔ صرافی جایگزین می‌شود. کندلهای بیرون پنجره حفظ می‌شوند تا عمق تاریخچه از دست نرود.

چرا این‌طوری؟ چرا کندل قدیمی داخل پنجره حذف می‌شود؟ چون اگر فقط «اضافه» می‌کردیم، یک کندل با قیمت خراب که زمانش تکراری نبود، از تعمیر جان سالم به در می‌برد. نسخهٔ تازهٔ صرافی همیشه مرجع است.

۸-۳. حالت «تاریخی» — یک تمایز حیاتی

وقتی داریم ۲ سال تاریخچه دانلود می‌کنیم، رفتار سختگیرانه فاجعه‌بار است. پس تابع اعتبارسنجی یک پارامتر `historical=True` دارد که چهار نوع مشکل را از «بحرانی» به «هشدار» تنزل می‌دهد:

کد مشکل	چرا در تاریخچه طبیعی است
EXCESSIVE_ZERO_VOLUME	آلت‌کوب‌های کم‌عمق در سال‌های اول واقعاً کندلهای بدون معامله دارند.
RETURN_OUTLIER	پامپ و کرش‌های واقعی در تاریخ کریپتو وجود داشته‌اند.
STALE_DATA	وقتی داری بازهٔ گذشته را می‌گیری، طبیعتاً «کهنه» است.
MISSING_CANDLES	صرافی در گذشته واقعاً قطعی داشته است.

⚠ هشدار: فقط خرابی‌های **ساختاری** (منطق OHLC شکسته، قیمت غیرمثبت، زمان ناهم‌تراز، تکراری، تاریخ آینده) در حالت تاریخی هم بحرانی می‌مانند. در چرخهٔ زنده هیچ تنزلی وجود ندارد و همهٔ قوانین سفت و سخت‌اند — چون یک کندل خراب همین‌لان باید جلوی

این تمایز نتیجه یک باگ واقعی است: مسیر سختگیرانه قدیمی، پنجره‌های عظیم را چهار بار دوباره دانلود می‌کرد و در نهایت هیچ چیز ذخیره نمی‌کرد. (کامیت‌های aaa13c5 و f545744)

۸-۴. بررسی‌های جانبی

دفتر سفارش و متریک‌های آتی هم بررسی می‌شوند، ولی همه مشکلاتشان فقط هشدار است: نبود دفتر سفارش باعث می‌شود ویژگی‌های ریزساختار خنثی شوند، نه اینکه چرخه متوقف شود.

۹. مرحله ۴: انبار داده — پایگاه داده SQLite

👤 به زبان بچه‌گانه: همه داده‌ها در یک فایل به اسم data/quant.db ذخیره می‌شوند. مثل یک دفتر بزرگ با ۶ بخش. هیچ سرور دیتابیس جداگانه‌ای لازم نیست — همین یک فایل کافی است، که برای یک سرور مجازی کوچک ایده‌آل است.

۹-۱. شش جدول (db_models.py)

جدول	محتوا
ohlcv	کندل‌های تأییدشده. کلید یکتا: (نماد، تایم‌فریم، زمان).
order_book_snapshots	آمار خلاصه دفتر سفارش. کلید یکتا: (نماد، زمان).
futures_metrics	فاندینگ، بهره‌ی باز، نسبت‌ها، لیکوئیدیشن‌ها.
audit_logs	هر تصمیم با ۳۰+ ستون: تصویر ویژگی‌ها، خروجی هر چهار مدل، قانون فعال‌شده.
trades	معاملات کاغذی و واقعی از باز شدن تا بسته شدن.
equity_curve	منحنی سرمایه، یک نقطه در هر چرخه.
system_state	ذخیره‌گاه کلید/مقدار پایدار: وضعیت نگهدارنده ریسک، انتخاب ارزها، لیست ارزهای آموزش، موجودی کاغذی.

۹-۲. سه تصمیم مهندسی مهم

الف) همه‌چیز غیرهمزمان است

حتی یک فراخوانی همزمان به پایگاه داده وجود ندارد. یعنی چرخه ۵ دقیقه‌ای و پنل وب هرگز پشت دیسک منتظر نمی‌مانند.

ب) نوشتن ایدمپوتنت (تکرارپذیر بدون عارضه)

```
INSERT ... ON CONFLICT (symbol, timeframe, timestamp) DO UPDATE SET ...
```

اجرای دوباره یک چرخه یا ذخیره مجدد یک بلوک تعمیرشده، هرگز ردیف تکراری نمی‌سازد.

ج) دو باگ واقعی که اینجا رفع شده‌اند

مشکل	راه‌حل
سقف پارامترهای SQLite	یک بک‌فیل عمیق ۱۰۰ هزار کندل را یک‌جا می‌داد؛ با ۸ ستون یعنی ۸۰۰ هزار پارامتر و شکست کامل. حالا در تکه‌های ۱۰۰۰ ردیفی نوشته می‌شود. (کامیت e67425d)
«database is locked»	SQLite فقط یک نویسنده در لحظه قبول می‌کند. وقتی بک‌فیل روی ۳۰ ارز پخش شده بود و همزمان چرخه ۵ دقیقه‌ای هم می‌نوشت، تراکنش‌ها به هم می‌خوردند. حالا یک asyncio.Lock درون‌پروژه‌ای همه تراکنش‌های نوشتن را سریال می‌کند. (کامیت d2c5c49)

۹-۳. تنظیمات PRAGMA

تنظیم	چرا
journal_mode=WAL	حالت «ثبت پیشاپیش»: پنل وب می‌تواند بخواند در حالی که ربات می‌نویسد.
busy_timeout=10000	اگر فایل قفل بود، ۱۰ ثانیه صبر کن به جای اینکه فوراً خطا بدهی. این روی هر اتصال استخر اعمال می‌شود، نه فقط اولی.
synchronous=NORMAL	تعادل بین سرعت و ایمنی داده.

foreign_keys=ON	یکپارچگی ارجاعی.
-----------------	------------------

۹-۴. بهینه‌سازی مسیر خواندن

برای گرفتن «۳۰۰۰ کندل آخر»، کد این کار را می‌کند:

```
SELECT ... ORDER BY timestamp DESC LIMIT 3000
-- استفاده از ایندکس، سپس در حافظه مرتب می‌شود --
```

؟ چرا این‌طوری؟ اگر `ORDER BY timestamp ASC` می‌نوشتیم و بعد `LIMIT` می‌زدیم، کندل‌های قدیمی‌ترین برمی‌گشتند. و اگر می‌خواستیم آخری‌ها را از مرتب‌سازی صعودی بگیریم، دیتابیس مجبور می‌شد کل جدول (میلیون‌ها ردیف) را اسکن کند. مرتب‌سازی نزولی `LIMIT +` یک «جست‌وجوی ایندکس» است، نه اسکن جدول.

۹-۵. متدهای اصلی

کار	متد
درج/به‌روزرسانی دسته‌ای کندل‌ها.	<code>upsert_candles()</code>
خواندن کندل‌ها به‌صورت جدول <code>pandas</code> با شاخص زمانی UTC.	<code>load_ohlcv_dataframe()</code>
جدیدترین و قدیمی‌ترین کندل ذخیره‌شده — پایهٔ بک‌فیل هوشمند.	<code>latest / earliest_candle_timestamp()</code>
شمارش کندل‌های یک ارز.	<code>candle_count()</code>
درج دسته‌ای رکوردهای ثبت وقایع در یک تراکنش.	<code>insert_audit_logs()</code>
ثبت باز شدن و بسته شدن معامله.	<code>insert_trade() / update_trade()</code>
ذخیره‌گاه کلید/مقدار پایدار (JSON).	<code>set_state() / get_state()</code>
حذف رکوردهای قدیمی‌تر از N روز.	<code>purge_audit_logs()</code>

۱. خط لوله مازول pipeline.py — A

این فایل تنها چیزی است که main.py از مازول A لازم دارد بشناسد. دو کار می‌کند.

۱.۰-۱. پر کردن تاریخچه (bootstrap_history)

برای هر ارز، ابتدا محاسبه می‌شود که چه چیزی کم است:

```
end = last_closed_candle_open_ms()
default_start = end - (210240 × 5m)
newest = latest_candle_timestamp(symbol)
oldest = earliest_candle_timestamp(symbol)
# پوشش ۲ سال گذشته : هدف
```

سپس دو بازه احتمالی ساخته می‌شود:

بازه	شرط	هدف
تاریخچه قدیمی‌تر	$oldest - 5m \geq default_start$	از default_start تا قبل از قدیمی‌ترین کندل موجود.
دنباله رو به جلو	$newest + 5m \leq end$	از بعد از جدیدترین کندل موجود تا حالا.

⚠ **هشدار:** یک باگ مهم که رفع شده: نسخه قدیمی فقط «دنباله رو به جلو» را می‌گرفت. یعنی اگر جدولت ۱۰۰۰ کندل داشت و بعد هدف را به ۲ سال زیاد می‌کردی، هیچ‌وقت عمیق‌تر نمی‌شد — تا ابد کم‌عمق می‌ماند. حالا هر دو سمت پوشش داده می‌شود. (کامیت d7e7c9f)

هر بازه جداگانه دانلود، با حالت **تاریخی** اعتبارسنجی، و ذخیره می‌شود. اگر QC مشکل ساختاری پیدا کند، هشدار داده می‌شود ولی کندل‌های سالم باز هم ذخیره می‌شوند.

همهٔ ارزها با `asyncio.gather` موازی پردازش می‌شوند، اما تحت یک `Semaphore`. شکست یک ارز، بقیه را متوقف نمی‌کند.

۱.۰-۲. چرخه زنده (run_cycle)

برای هر ارز، تابع `_process_symbol()` این‌ها را انجام می‌دهد:

۱ سه درخواست موازی: کندل‌ها، دفتر سفارش، متریک‌های آتی (با `return_exceptions=True` تا شکست یکی بقیه را نکشد).

۲ اگر کندل‌ها شکست خوردند ← این ارز `None` برمی‌گرداند و حذف می‌شود. اگر فقط دفتر سفارش یا متریک شکست خورد ← هشدار و ادامه با مقدار `None`.

۳ عبور از پلیس کیفیت با تعمیر خودکار.

۴ بررسی‌های جانبی و ساخت گزارش کیفیت غنی‌شده.

۵ ذخیره در پایگاه داده.

۶ برگرداندن یک `MarketDataBundle`.

در پایان، فقط ارزهایی که `is_valid` هستند در دیکشنری خروجی می‌مانند. ارزی که رد شده، اصلاً در خروجی نیست — پس مازول‌های بعدی حتی نمی‌بینندش.

۱۰-۳. تابع `required_lookback_bars()`

حداقل تاریخچه‌ای که مازول B لازم دارد:

```
max(garch_window=500, hmm_window=1000, rank_window=288,  
    fdi_window=30, kama_slow=30, atr_window=14) + 50  
= 1050 bars
```

یعنی برای هر ارز، دست‌کم حدود ۱۰۵۰ کندل (حدود ۳۶ روز) لازم است تا ویژگی‌ها گرم شوند.

۱۱. مرحله ۵: ساخت ویژگی‌ها — features.py و indicators.py

👉 **به زبان بچه‌گانه:** تا اینجا فقط چهار عدد خام داریم: باز، بالا، پایین، بسته. ولی مدل با عدد خام چیزی یاد نمی‌گیرد. باید از این چهار عدد، سؤال‌های معنی‌دار بسازیم: «روند بالاست یا پایین؟» «بازار الان آرام است یا دیوانه؟» «حجم غیرعادی است؟» هر جواب یک «ویژگی» است. در پایان ۲۶ ویژگی ساخته می‌شود.

⚠️ **هشدار:** قانون طلایی این بخش: مقدار هر ویژگی در کندل t فقط و فقط می‌تواند از داده‌های کندل t و قبل‌تر ساخته شود. هیچ پنجره متمرکز (center=True)، هیچ پرکردن رو به عقب (backward fill)، هیچ shift(-k). این «علی بودن» نام دارد و نقص آن، تنها اشتباهی است که یک بک‌تست سودآور را به یک سیستم زبان‌ده در دنیای واقعی تبدیل می‌کند.

۱۱-۱. اندیکاتورهای پایه (indicators.py)

همه از صفر با NumPy/pandas نوشته شده‌اند. کتابخانه ta-lib عمداً استفاده نشده چون یک افزونه C کامپایل‌شده لازم دارد که روی یک VPS کوچک دردسر است.

محدوده واقعی (True Range) و ATR

```
TR = max( high - low, |high - prev_close|, |low - prev_close| )
ATR = wilder_smooth(TR, window)
# alpha = 1/window میانگین نمایی با = هموارسازی وایلدِر
```

👉 **به زبان بچه‌گانه:** ATR یعنی «معمولاً این ارز در ۵ دقیقه چقدر تکان می‌خورد؟» اگر ATR بیت‌کوین ۱۰۰ دلار و ATR یک آلت‌کوین ۰.۰۰۱ دلار باشد، هر دو ممکن است به یک اندازه «پرنوسان» باشند نسبت به قیمت خودشان. ATR واحد سنجش نوسان است و همه حد سود و ضررها بر مبنای آن ساخته می‌شوند.

RSI (شاخص قدرت نسبی)

سودها و ضررهای هر کندل جدا می‌شوند، هر کدام با روش وایلدِر هموار می‌شوند، و نسبتشان به عدد ۰ تا ۱۰۰ تبدیل می‌شود. کد حالت‌های خاص را هم می‌پوشاند: اگر هیچ ضرری نبوده ← ۱۰۰؛ اگر هیچ سودی نبوده ← ۰ (به جای بی‌نهایت).

KAMA — میانگین متحرک تطبیقی کافمن

```
ER = |close_t - close_{t-n}| / sum(|close_i - close_{i-1}|)
SC = ( ER * (2/(fast+1) - 2/(slow+1)) + 2/(slow+1) )^2
KAMA_t = KAMA_{t-1} + SC * (close_t - KAMA_{t-1})
# ER = نسبت کارایی، SC ثابت هموارسازی
```

👉 **به زبان بچه‌گانه:** «نسبت کارایی» (ER) می‌پرسد: «برای رسیدن از A به B، مسیر مستقیم رفتی یا زیگزاگ؟» اگر مستقیم رفته باشد (روند تمیز)، ER نزدیک ۱ می‌شود و KAMA تقریباً چسبیده به قیمت حرکت می‌کند. اگر زیگزاگ رفته باشد (بازار درهم)، ER نزدیک ۰ می‌شود و KAMA صاف و بی‌حرکت می‌ماند. یعنی KAMA خودش می‌فهمد کی باید حساس باشد و کی نباید فریب نویز را بخورد.

این بازگشتی است و نمی‌شود کاملاً برداری‌اش کرد، پس با یک حلقه NumPy با پیچیدگی $O(n)$ نوشته شده — که به مراتب سریع‌تر از rolling.apply است.

FDI — شاخص بُعد فراکتالی

```
# در هر پنجره قیمت‌ها به بازه صفر تا یک نرمال می‌شوند
path_length = sum( sqrt( dy^2 + (1/n)^2 ) )
FDI = 1 + ( ln(path_length) + ln(2) ) / ln(2n)
```

مقدار FDI	معنی
حدود ۱۰	خط صاف — روند خالص و تمیز.
حدود ۱۵	گشت تصادفی — هیچ ساختار قابل بهره‌برداری نیست.
حدود ۲۰	مسیر تمام صفحه را پر کرده — بازار درهم و پرنوسان.

👉 **به زبان بچه‌گانه:** تصور کن یک مورچه از گوشهٔ چپ کاغذ به گوشهٔ راست می‌رود. اگر مستقیم برود، یک «خط» است (بُعد ۱). اگر آن‌قدر بالا و پایین برود که تمام کاغذ را سیاه کند، دیگر خط نیست، «سطح» است (بُعد ۲). FDI دقیقاً همین را اندازه می‌گیرد.

از FDI یک ویژگی دودویی هم ساخته می‌شود: $\text{fdi_trending} = (\text{fdi} < 1.5)$ — یعنی «آیا بازار روند دارد؟»
محاسبه با `sliding_window_view` کاملاً برداری است. پنجرهٔ کاملاً صاف (بدون تغییر قیمت) به عنوان گشت تصادفی (۱/۵) در نظر گرفته می‌شود.

ADX و DI

+DM و -DM از حرکت سقف و کف محاسبه می‌شوند، با ATR نرمال شده و هموار می‌شوند تا +DI و -DI بدهند. ADX قدرت روند (بدون توجه به جهت) را می‌سنجد. ویژگی $\text{di_spread} = (+DI - -DI) / 100$ جهت روند را می‌دهد.

باند‌های بولینگر

میانگین متحرک ± 2 انحراف معیار. سه خروجی: خط میانی، عرض باند نسبی، و موقعیت قیمت داخل باند (b%). در مجموعهٔ نهایی ویژگی‌های مدل استفاده نمی‌شود ولی محاسبه می‌شود.

تبدیل‌های ایستا (Stationary)

تابع	کار
<code>rolling_percentile_rank()</code>	«این مقدار نسبت به ۲۸۸ کندل گذشتهٔ خودش چقدر بزرگ است؟» خروجی بین ۰ و ۱.
<code>rolling_zscore()</code>	چند انحراف معیار از میانگین متحرک فاصله دارد.
<code>slope()</code>	تغییر N کندلی تقسیم بر سطح فعلی — قابل مقایسه بین بیت‌کوین و پیه.

❓ **چرا این‌طوری؟** چرا «رتبهٔ درصدی» به جای مقدار خام؟ چون درخت‌های تصمیم روی مقدارهای مطلق غیرایستا تعمیم نمی‌دهند. اگر مدل یاد بگیرد «حجم بالای ۵ میلیون یعنی خطر»، شش ماه بعد که کل بازار ۱۰ برابر شده، آن قانون بی‌معنی است. اما «حجم در صدک ۹۵ خودش» همیشه معنی یکسانی دارد. پنجرهٔ ۲۸۸ کندلی دقیقاً ۲۴ ساعت است.

۱۱-۲. ویژگی‌های پیشرفته (features.py)

الف) بازدهی و مومنتوم

```
log_return_1 = log(close).diff(1)    # دقیقه ۵
log_return_3 = log(close).diff(3)    # دقیقه ۱۵
log_return_12 = log(close).diff(12)  # ساعت ۱
momentum_rank = rolling_percentile_rank(log_return_12, 288)
```

ب) بستر روند

```
kama_distance = (close - KAMA) / close  # فاصله از میانگین تطبیقی
kama_slope    = slope(KAMA, 5)          # جهت روند
adx           = slope(ADX, 5)           # قدرت روند
di_spread     = (+DI - -DI) / 100       # سمت روند
```

ج) بستر تایم فریم بالاتر – جایی که بیشتر لبه جهتی از آن می‌آید

همه از روی همان سری ۵ دقیقه‌ای ساخته می‌شوند (۱۲ کندل = ۱ ساعت، ۴۸ کندل = ۴ ساعت). هیچ نمونه‌برداری مجددی انجام نمی‌شود، پس علی بودن حفظ می‌شود.

ویژگی	معنی
htf_ema_slope_1h	شیب میانگین نمایی ۱ ساعته، نرمال شده با قیمت.
htf_ema_slope_4h	شیب میانگین نمایی ۴ ساعته.
htf_close_vs_ema_4h	فاصله قیمت فعلی از میانگین ۴ ساعته.
htf_return_4h	بازدهی لگاریتمی ۴۸ کندلی.
htf_trend_align	$\text{sign}(\text{slope_1h}) \times \text{sign}(\text{slope_4h})$: ۱ هر دو صعودی، -۱ هر دو نزولی، ۰ اختلاف نظر.

👤 **به زبان بچه‌گانه:** مثل این است که وقتی داری در یک کوچه راه می‌روی، سرت را بلند کنی و ببینی کل خیابان به کدام سمت شیب دارد. یک ربات ۵ دقیقه‌ای که فقط ۵ دقیقه بعدی را می‌بیند، مثل کسی است که با چشم روی زمین راه می‌رود.

د) نوسان

```
atr_pct = ATR / close # ویژگی مدل
atr_rank = rolling_percentile_rank(atr_pct, 288) # فقط برای برچسب‌زنی
```

ه) حجم

```
volume_zscore = rolling_zscore(volume, 288)
volume_trend = mean(volume, 12) / mean(volume, 96) - 1
```

و) GARCH – پیش‌بینی نوسان فردا

👤 **به زبان بچه‌گانه:** GARCH یک قانون ساده دارد: «نوسان خوشه‌ای است». روز شلوغ بعد از روز شلوغ می‌آید، روز آرام بعد از روز آرام. پس با نگاه به نوسان دیروز، می‌شود نوسان فردا را حدس زد.

رابطه بازگشتی $GARCH(1,1)$:

$$\sigma^2_{t+1} = \omega + \alpha \times \epsilon^2_t + \beta \times \sigma^2_t$$

روش کار در کد:

- مدل هر ۵۰ کندل روی پنجره ۵۰۰ کندل اخیر دوباره برازش می‌شود.
- بین دو برازش، همان رابطه بالا با بازدهی‌های واقعی جلو برده می‌شود — که هم علی است و هم ارزان.
- خروجی در کندل t پیش‌بینی یک قدم جلوتر است، که طبق تعریف در لحظه t معلوم است.
- بازدهی‌ها $100 \times$ می‌شوند (کتابخانه arch با درصد بهتر کار می‌کند) و نتیجه دوباره تقسیم می‌شود.

محافظت‌ها:

- اگر پارامترها بی‌معنی باشند ($\omega \leq 0$ یا $\alpha + \beta \geq 1$) که یعنی مدل غیرایستا است، برازش رد می‌شود و برازش قبلی نگه داشته می‌شود.
- اگر کتابخانه arch نصب نباشد یا برازش شکست بخورد، به نوسان EWMA (سیک RiskMetrics) تنزل پیدا می‌کند.

چرا این طوری؟ چرا تنزل به EWMA به جای برگرداندن NaN؟ چون یک ویژگی ضعیف تر قابل بازیابی است، اما یک حفره NaN باعث می شود آن ردیف کاملاً حذف شود و ممکن است کل ارز از دور خارج شود.

ن) HMM – تشخیص رژیم بازار

👤 **به زبان بچه گانه:** بازار «حالت» های مختلفی دارد: گاوی (صعودی)، خرسی (نزولی)، پرنوسان، و راکد. مدل مارکوف پنهان سعی می کند بفهمد بازار الان در کدام حالت است – بدون اینکه کسی به او گفته باشد این حالت ها چه هستند.

سه کانال مشاهده که به مدل داده می شود:

کانال	چه چیزی را جدا می کند
بازدهی علامت دار	جهت (گاوی از خرسی)
$\log(\text{high/low})$	نوسان و اوره شدگی
حجم نسبی	میزان مشارکت

دو نکته فنی حیاتی در پیاده سازی HMM

۱ فقط فیلتر رو به جلو (آلفا)، نه ویتربی و نه هموارسازی.

الگوریتم های معمول HMM (مثل ویتربی یا forward-backward) برای تعیین حالت در لحظه t از داده های $t+1$ تا انتها هم استفاده می کنند. این یعنی **نگاه کردن به آینده**. کد اینجا فقط بازگشت آلفا را در فضای لگاریتمی اجرا می کند که فقط از x_{t-1} تا x_t استفاده می کند.

۲ نگاشت به یک تاکسونومی پایدار.

کتابخانه `hmmlearn` به حالت ها شماره های دلخواه می دهد، و این شماره ها در هر برازش مجدد عوض می شوند! پس شماره خام کاملاً بی معنی است. تابع `canonical_state_map()` از روی میانگین های انتشار، حالت ها را به چهار رژیم ثابت نگاشت می کند:

- حالتی که بیشترین میانگین دامنه را دارد ← **پرنوسان** (۲).
- از بقیه، بیشترین میانگین بازدهی ← **روند گاوی** (۰).
- از بقیه، کمترین میانگین بازدهی ← **روند خرسی** (۱).
- باقی مانده ← **راکد** (۳).

چرا این طوری؟ بدون این نگاشت، ویژگی «رژیم» در هر برازش معنی اش عوض می شد و مدل فقط نویز یاد می گرفت.

شش ویژگی از HMM بیرون می آید:

ویژگی	معنی
<code>hmm_regime</code>	رژیم غالب فعلی (۰ تا ۳، یا -۱ قبل از گرم شدن).
<code>hmm_prob_bull</code>	احتمال فیلتر شده رژیم گاوی.
<code>hmm_prob_bear</code>	احتمال رژیم خرسی.
<code>hmm_prob_high_vol</code>	احتمال رژیم پرنوسان.
<code>hmm_prob_sideways</code>	احتمال رژیم راکد.
<code>hmm_regime_age</code>	چند کند است که رژیم فعلی ادامه دارد، تقسیم بر ۴۸ و با سقف ۵.

تابع چگالی گاوسی قطری دستی پیاده‌سازی شده تا فیلتر به جزئیات داخلی hmmlearn که بین نسخه‌ها عوض می‌شوند وابسته نباشد. همچنین توابع `_log_sum_exp` و `_log_matmul` برای پایداری عددی در فضای لگاریتمی نوشته شده‌اند.

ح) ریزساختار – محاسبه می‌شود ولی وارد مدل نمی‌شود

دفتر سفارش و متریک‌های آتی با `merge_asof(direction="backward")` به کندل‌ها چسبانده می‌شوند. جهت «backward» یعنی هر کندل فقط با اسنپ‌شاتی جفت می‌شود که زمانش کمتر یا مساوی زمان خودش است — همان اصل علی بودن.

⚠ هشدار: چرا این ویژگی‌ها به مدل داده نمی‌شوند؟ چون بایننس هیچ فید تاریخی عمیقی از دفتر سفارش یا لیکوئیدیشن ارائه نمی‌دهد. یعنی در دیتاست آموزشی که از تاریخچه ساخته می‌شود، این ستون‌ها همه ثابت هستند. اگر مدل روی ستون ثابت آموزش ببیند و بعد در زنده مقدار متغیر بگیرد، یک انحراف آموزش/سرویس ایجاد می‌شود که به مدل ضربه می‌زند. پس این‌ها فقط برای نمایش در پنل و قواعد جایگزین مدل‌ها نگه داشته می‌شوند.

۱۱-۳. قرارداد نهایی: ۲۶ ویژگی مدل

ثابت `FEATURE_COLUMNS` یک تاپل مرتب است. ترتیب بخشی از قرارداد است: مدل ذخیره‌شده ویژگی‌ها را دقیقاً با همین چیدمان انتظار دارد، چون درخت‌ها ویژگی‌ها را با شماره (نه با اسم) می‌شناسند.

دسته	تعداد	ویژگی‌ها
روند / جهت	۴	kama_distance, kama_slope, adx, di_spread
ساختار	۲	fdi, fdi_trending
مومنتوم	۵	rsi, log_return_1, log_return_3, log_return_12, momentum_rank
تایم‌فریم بالاتر	۵	htf_ema_slope_1h, htf_ema_slope_4h, htf_close_vs_ema_4h, htf_return_4h, htf_trend_align
نوسان	۲	atr_pct, garch_volatility
رژیم	۶	hmm_regime, hmm_prob_bull, hmm_prob_bear, hmm_prob_high_vol, hmm_prob_sideways, hmm_regime_age
حجم	۲	volume_zscore, volume_trend
مجموع		۲۶ ویژگی

نکته: در کد ثابت `ARTIFACT_VERSION = "2.0.0"` این قرارداد ۲۶ ویژگی‌ای را علامت‌گذاری می‌کند. اگر فایل مدل ذخیره‌شده نسخه دیگری داشته باشد، هنگام بارگذاری رد می‌شود و مدل مجبور به آموزش دوباره است. (در توضیحات کد گاهی عدد ۲۱ ذکر شده که مربوط به یک نسخه میانی بوده؛ مقدار واقعی فعلی ۲۶ است.)

۱۱-۴. اجرای بدون بلوکه کردن (FeatureService)

ساخت ویژگی کار سنگین CPU است. اگر روی حلقه اصلی رویداد اجرا شود، پنل وب هنگ می‌کند. پس به‌صورت پیش‌فرض با `asyncio.to_thread` به یک نخ کارگر منتقل می‌شود (pandas و NumPy در کرنل‌های عددی سنگین GIL را آزاد می‌کنند، پس واقعاً موازی می‌شود). برای بک‌تست‌های عریض می‌شود `use_process_pool=True` گذاشت.

متد `build_many()` چند ارز را همزمان پردازش می‌کند و ارزی که شکست بخورد فقط لاگ می‌شود و از نتیجه حذف می‌شود — کل چرخه از بین نمی‌رود.

۱۲. مرحله ۶: معلم مدل — برچسب‌زنی (labeler.py)

🧠 به زبان بچه‌گانه: مدل باید یاد بگیرد. ولی از روی چه چیزی؟ ما باید به او بگوییم «در این لحظه، جواب درست این بود». برای این کار به گذشته برمی‌گردیم، از هر نقطه یک معامله فرضی باز می‌کنیم، و می‌بینیم چه شد.

⚠️ **هشدار:** این تنها فایل است که اجازه دارد به آینده نگاه کند — چون دارد «جواب صحیح» را می‌سازد. به همین دلیل کد آن را هرگز در مسیر پیش‌بینی زنده وارد نمی‌کند. فایل processor.py این جدایی را با دو تابع کاملاً مجزا تضمین می‌کند.

۱۲-۱. شبیه‌سازی سه‌مانعی (Triple Barrier)

در هر کندل، دو معامله فرضی همزمان باز می‌شود: یک لانگ و یک شورت.

```
tp_distance = 2.0 * ATR    # فاصله حد سود
sl_distance = 1.0 * ATR    # فاصله حد ضرر
horizon      = 48 bars     # افق ۴ ساعته
```

سه «مانع» وجود دارد و اولین مانعی که لمس شود، معامله را می‌بندد:

مانع	نتیجه
حد سود (بالا/پایین)	TAKE_PROFIT — برنده
حد ضرر (پایین/بالا)	STOP_LOSS — بازنده
مانع زمانی (۴۸ کندل)	EXPIRED — با قیمت پایان افق بسته می‌شود

۱۲-۲. سه انتخاب طراحی که برچسب‌ها را صادقانه نگه می‌دارند

۱. بدترین ترتیب ممکن داخل کندل.

داده ۵ دقیقه‌ای نمی‌گوید که اول سقف آمد یا اول کف. اگر یک کندل هم حد سود و هم حد ضرر را لمس کرده باشد، کد فرض می‌کند **حد ضرر اول خورده**.

```
stop_first = sl_any & (~tp_any | (first_sl <= first_tp))
```

❓ **چرا این‌طوری؟** هر فرض دیگری، «برنده خیالی» می‌سازد. اگر خوش‌بین باشی، بک‌تستت زیبا می‌شود و حساب واقعیات خالی.

۲. **خروج در قیمت مانع، نه قیمت بسته‌شدن.** معامله حل‌شده دقیقاً در قیمت مانع بسته می‌شود؛ فقط معامله منقضی‌شده در قیمت پایان افق ارزش‌گذاری می‌شود.

۳. «درد مسیر» اندازه‌گیری می‌شود، نه نادیده گرفته.

بیشترین حرکت خلاف جهت (MAE) در طول مسیر ثبت و به‌صورت کسری از فاصله حد ضرر بیان می‌شود.

👤 **به زبان بچه‌گانه:** دو معامله هر دو با سود بسته شده‌اند. ولی یکی از اول تا آخر در سود بوده، و آن یکی سه ساعت روی لبه حد ضرر می‌لرزیده. این دو معامله یکی نیستند — و ستون «رده ریسک» دقیقاً همین را می‌گوید.

۱۲-۳. اجرای برداری و کارآمد

شبیه‌سازی با `sliding_window_view` و `np.minimum.accumulate / np.maximum.accumulate` انجام می‌شود، یعنی کل تاریخچه در چند عبور آرایه‌ای حل می‌شود، نه با حلقه دوگانه پایتون. کار در تکه‌های ۲۰'۰۰۰ ردیفی انجام می‌شود تا مصرف حافظه محدود بماند. مقیاس مانع: اول ATR، اگر گرم نشده بود پیش‌بینی GARCH ضرر در قیمت، و در نهایت ۰.۲۵٪ قیمت به عنوان کف منطقی.

۱۲-۴. انتخاب برچسب نهایی (`_classify`)

- یک سمت فقط وقتی «برنده» است که **حد سودش** معامله را حل کرده باشد.
- اگر هر دو سمت برنده باشند (بازار اره‌ای که هر دو طرف را در افق زده)، سمتی که **زودتر** حل شده برنده است؛ در صورت تساوی، سمتی که **درد کمتری** داشته.
- رده ریسک برنده از روی MAE محاسبه و با رژیم نوسان تشدید می‌شود.
- اگر رده به `VERY_HIGH` رسید، برچسب به `NO_TRADE` تبدیل می‌شود.

جدول رده ریسک

نسبت MAE	رده پایه
≥ 0.35	LOW
0.35 تا 0.60	MEDIUM
0.60 تا 0.85	HIGH
< 0.85	VERY_HIGH

سپس تشدید بر اساس نوسان:

- صدک نوسان ≤ 0.95 ← مستقیماً `VERY_HIGH`.
- صدک نوسان ≤ 0.80 ← یک پله بالاتر.

❓ **چرا این‌طوری؟** چرا نوسان رده را بالا می‌برد؟ چون همان مقدار «درد» در بازاری که کندل بعدی‌اش می‌تواند سه برابر بزرگ‌تر باشد، به مراتب خطرناک‌تر است.

۱۲-۵. سه کلاس هدف، نه پنج کلاس

🔥 **نکته:** در نسخه قبلی این پروژه، هدف جهت پنج کلاس داشت (`LONG_LOW_RISK`, `LONG_HIGH_RISK`, `SHORT_LOW_RISK`, ...). مشکل: سطوح `*_HIGH_RISK` خیلی کوچک و پر از نویز بودند و یادگیری جهت را به شدت سخت می‌کردند. حالا هدف جهت فقط **سه کلاس متوازن** دارد: `LONG`, `SHORT`, `NO_TRADE` — و اطلاعات ریسک به صورت جداگانه در ستون `risk_tier` و هدف مدل ریسک حمل می‌شود. (کامیت 912d0f6)

۱۲-۶. اهداف هر چهار مدل

مدل	ستون هدف	چگونه ساخته می‌شود
جهت	<code>label / label_index</code>	سه کلاس <code>LONG / SHORT / NO_TRADE</code>

ورود	entry_quality	دودویی: ۱ فقط وقتی سمت انتخابی برنده شده و هیچ وقت بیش از ۰.۳۵ فاصله حد ضرر عقب نرفته. این دقیقاً سؤال «الان وارد شوم یا یک کندل صبر کنم» است.
خروج	target_tp_pct target_sl_pct target_trailing_pct	از هندسه واقعی حرکت معامله: بهترین قیمتی که در افق به آن رسیده (۰.۹×) برای جا گذاشتن برای لغزش و گرمایی که تا آن نقطه تحمل شده (۱.۱۵×). حد ضرر هرگز کمتر از نصف فاصله نوسان محور نمی شود. تریلینگ = نصف حد سود.
ریسک	target_risk_score	(صدک نوسان - 1) × 0.5 + 0.5 × (1 - MAE) ، محدود به بازه ۰ تا ۱.

۱۲-۷. ستون label_is_valid

۴۸ کندل آخر هر ارز قابل شبیه سازی نیستند (چون آینده شان هنوز نیامده). این ستون آن ها را False علامت می زند و پردازشگر آن ها را از دیتاست آموزشی حذف می کند.

۱۳. مرحله ۷: ساخت دفترچه تمرین — processor.py

این فایل دو در ورودی کاملاً جدا دارد، و این جدایی عمدی است:

تابع	حالت	برچسب زن؟
build_training_dataset()	تاریخی/آموزش	بله — با نگاه به آینده
build_inference_payload()	زنده	خیر — اصلاً وارد این مسیر نمی‌شود

۱۳-۱. مسیر آموزش

۱ برای هر ارز: خواندن کندل‌ها + متریک‌های آتی + دفتر سفارش از پایگاه داده.

۲ ساخت ویژگی‌ها (روی نخ کارگر).

۳ اجرای برچسب زن (روی نخ کارگر).

۴ افزودن ستون symbol و ادغام همه ارزش‌ها در یک جدول واحد.

۵ مرتب‌سازی کل جدول بر اساس زمان (نه بر اساس ارز).

چرا این‌طوری؟ چرا همه ارزش‌ها با هم و مرتب بر اساس زمان؟ چون آن‌وقت تقسیم آموزش/آزمون یک تقسیم واقعی خارج-از-زمان است. اگر هر ارز جدا تقسیم می‌شد، مدل می‌توانست از رفتار بیت‌کوین در آینده برای پیش‌بینی گذشته اتریوم تقلب کند.

۶ پاک‌سازی: حذف ردیف‌های `label_is_valid == False`، تبدیل بی‌نهایت به NaN، حذف ردیف‌هایی که هر ویژگی یا برچسبشان NaN است (این‌ها همان ردیف‌های «گرم‌شدن» پنجره‌های متحرک هستند).

۷ جدا کردن به: ماتریس ویژگی، هدف جهت، هدف ورود، اهداف خروج، هدف ریسک، و ابرداده.

۱۳-۲. تقسیم پاک‌سازی‌شده — مهم‌ترین جزئیات آماری پروژه

```
rows = total_rows
split_point = rows * (1 - 0.2)
train_end = split_point - 60 # این ۶۰ ردیف کاملاً حذف می‌شوند
train = rows[0 : train_end]
validation = rows[split_point : end]
```

```
|<----- آموزش ----->| حذف |<--- آزمون --->|
0                train_end split      rows
                \___/ ۶۰ ردیف پاک‌سازی
```

چرا این‌طوری؟ چرا آن ۶۰ ردیف حذف می‌شوند؟ این نکته را اکثر پروژه‌ها اشتباه می‌کنند. برچسب هر ردیف از آینده آن ردیف ساخته شده (تا ۴۸ کندل جلوتر). پس برچسب آخرین ردیف‌های بلوک آموزش، با قیمت‌های بلوک آزمون هم‌پوشانی دارد. اگر این ردیف‌ها

بمانند، مدل به‌طور غیرمستقیم جواب امتحان را دیده و نمرهٔ آزمون به‌طور ساختاری خوش‌بینانه می‌شود. حذف ۶۰ ردیف (بیشتر از ۴۸ کندل افق) این نشت را می‌بندد.

۱۳-۳. مسیر زنده

1. عمق لازم محاسبه می‌شود: $1220 = 120 + 100 + \max(500, 1000, 288)$ کندل.
2. ویژگی‌ها ساخته می‌شوند (بدون هیچ برچسبی).
3. ردیف‌هایی که هنوز NaN دارند حذف می‌شوند.
4. **یک بررسی حیاتی:** اگر آخرین ردیف کامل با آخرین ردیف موجود فرق داشته باشد، یعنی ویژگی‌های جدیدترین کندل هنوز گرم نشده‌اند. در این حالت آن ارز رد می‌شود.

? چرا این‌طوری؟ چرا این بررسی؟ چون در غیر این صورت مدل روی یک کندل کهنه (مثلاً ۳ کندل قبل) تصمیم می‌گرفت و آن را به قیمت الان اعمال می‌کرد. این یک نوع سکتۀ خاموش است.

خروجی یک InferencePayload است: نماد، زمان، قیمت بسته، یک ردیف ویژگی، و یک «تصویر لحظه‌ای» خوانا برای دفتر ثبت وقایع (شامل رژیم HMM، نوسان GARCH، شیب RSI، ADX، ATR، FDI، KAMA، عدم توازن دفتر سفارش، اسپرد، نرخ فاندینگ، z-score حجم و قیمت).

۱۴. مرحله ۸: چهار مغز کوچک — ml_models.py

👤 **به زبان بچه‌گانه:** به جای یک مدل غول‌پیکر که همه چیز را با هم تصمیم بگیرد، چهار متخصص داریم. هر کدام فقط یک سؤال بلد است و از وجود بقیه خبر ندارد. مثل یک تیم پزشکی: یکی می‌گوید بیماری چیست، یکی می‌گوید الان عمل کنیم یا فردا، یکی می‌گوید چه دارویی، و یکی می‌گوید چقدر دوز.

مدل	سؤال	نوع	خروجی
۱ جهت	بازار کجا می‌رود؟	طبقه‌بند سه‌کلاسه	توزیع احتمال روی LONG / SHORT / NO_TRADE
۲ ورود	الان یا یک کندل صبر کنیم؟	طبقه‌بند دودویی	احتمال + بله/خیر
۳ خروج	معامله چگونه مدیریت شود؟	سه رگرسیون	درصد حد سود، حد ضرر، فعال‌سازی تریلینگ
۴ ریسک	چقدر پول و اهرم؟	رگرسیون + منطق قطعی	اهرم ۰ تا ۱۰ + درصد تخصیص سرمایه

۱۴-۱. زیرساخت مشترک (BaseModelHead)

الگوریتم

پیش‌فرض **LightGBM** است (با گزینه XGBoost). این‌ها «درخت‌های تصمیم تقویت‌شده گرادینتی» هستند: صدها درخت کوچک که هر کدام خطای درخت‌های قبلی را کمی اصلاح می‌کند.

پارامترهای ضد حفظ‌کردن:

- subsample=0.85 — هر درخت فقط ۸۵٪ ردیف‌ها را می‌بیند.
- colsample_bytree=0.85 — هر درخت فقط ۸۵٪ ستون‌ها را می‌بیند.
- min_child_samples=40 — هر برگ حداقل ۴۰ نمونه داشته باشد.
- reg_lambda=1.0 — جریمه پیچیدگی.
- class_weight="balanced" — اگر یک کلاس کم‌تعداد باشد، وزنش بالا می‌رود تا مدل آن را نادیده نگیرد.

توقف زودهنگام

اگر ۵۰ دور پیاپی روی بلوک آزمون بهتر نشد، آموزش قطع می‌شود. یک نکته ظریف مهندسی: LightGBM نسخه ۴٫۷ نام پارامترها را از eval_set به eval_x / eval_y تغییر داد. کد با inspect.signature تابع را بررسی می‌کند و با هر دو نسل کتابخانه کار می‌کند.

هم‌ترازی ویژگی‌ها (align_) — یک سپر حیاتی

درخت‌ها ویژگی‌ها را با موقعیت می‌شناسند، نه با اسم. اگر ستون‌ها بی‌سروصدا جابه‌جا شوند، مدل «مزخرف با اطمینان بالا» تولید می‌کند. پس قبل از هر پیش‌بینی، ورودی دقیقاً روی چیدمان زمان آموزش reindex می‌شود، ستون‌های گم‌شده با ۰٫۰ پر و لاگ می‌شوند، و بی‌نهایت/NaN به ۰ تبدیل می‌شود.

ذخیره و بارگذاری

هر مدل با joblib (فشرده‌سازی سطح ۳) ذخیره می‌شود، همراه با: نسخه آرتیفکت، نام مدل، خود مدل، لیست دقیق ستون‌ها، و ابرداده (زمان آموزش، تعداد ردیف، معیارها).

🔴 **نکته:** تابع load_artifact() هرگز خطا پرتاب نمی‌کند. اگر فایل نبود، خراب بود، یا نسخه‌اش با ARTIFACT_VERSION = "2.0.0" نمی‌خواند، فقط None برمی‌گرداند و آن مدل به قاعده جایگزین تنزل پیدا می‌کند. یک مدل گم‌شده باید سیستم را به حالت «معامله نکن» ببرد، نه اینکه حلقه معاملاتی را بکشد.

۱۴-۲. مدل ۱: جهت (DirectionModel)

یک طبقه‌بند سه‌کلاسه. برچسب‌ها با نگاشت ثابت LABEL_ORDER = (LONG, SHORT, NO_TRADE) کدگذاری می‌شوند.

معیارهای ارزیابی: دقت، دقت متوازن (که کلاس‌های کم‌تعداد را هم می‌شمارد)، و log-loss.

قاعده جایگزین (وقتی مدل وجود ندارد)

```
score = 60 × kama_slope + 1.2 × di_spread
if regime == BULL: score += 0.25
if regime == BEAR: score -= 0.25
directional = sigmoid(score) # بین صفر و یک
trade_mass = 0.45 × (0.35 + 0.65 × fdi_trending)
long = trade_mass × directional
short = trade_mass × (1 - directional)
no_trade = 1 - long - short
```

👉 به زبان بچه‌گانه: در بازار درهم (FDI بالای ۱٫۵)، ضریب trade_mass کوچک می‌شود و حداقل ۵۵٪ احتمال روی «کاری نکن» می‌ماند. یعنی قاعده جایگزین عمده‌اً محتاط است. و هر خروجی آن با برچسب HEURISTIC مهر می‌خورد.

۱۴-۳. مدل ۲: ورود (EntryModel)

طبقه‌بند دودویی روی entry_quality. معیارها: دقت مثبت (precision)، فراخوانی (recall)، و مساحت زیر منحنی ROC. اگر احتمال ≤ 0.5 باشد، جواب «الان وارد شو» است.

قاعده جایگزین

```
score = 0.5
score += directional_flow × 0.15 # عدم توازن دفتر سفارش هم‌جهت
score += (fdi_trending - 0.5) × 0.20 # ساختار روندی
score -= (spread_rank - 0.5) × 0.20 # جریمة اسپرد بالا
```

یعنی: جریان سفارش‌ها هم‌جهت باشد، بازار روند داشته باشد، و اسپرد در دهه بالای خودش نباشد.

۱۴-۴. مدل ۳: خروج (ExitModel)

سه رگرسیون مستقل برای حد سود، حد ضرر، و فعال‌سازی تریلینگ. فقط روی ردیف‌هایی آموزش می‌بیند که واقعاً یک معامله جهت‌دار انتخاب شده باشد — چون فقط آن ردیف‌ها هندسه معنادار دارند. اگر کمتر از ۱۰۰ ردیف باشد، خطا می‌دهد.

ریلهای ایمنی (_assemble) — روی هر خروجی، چه مدل چه جایگزین

قانون	مقدار
حد ضرر حداقل نصف ATR	<code>_MIN_SL_ATR_FRACTION = 0.5</code>
حد ضرر بین	۸٪ و ۱۵٪
حد سود بین	۲٪ و ۱۵٪
حد سود بین ۱٫۱ و ۶ برابر حد ضرر	<code>_MAX_REWARD_RISK = 6.0</code>
فعال‌سازی تریلینگ بین	۲۵٪ و ۹۵٪ حد سود
فاصله تریلینگ	۶۰٪ حد ضرر

چرا این‌طوری؟ یک رگرسور می‌تواند عدد منفی یا مسخره بدهد؛ صراف‌ی نمی‌تواند آن را قبول کند. دو ریل خیلی مهم‌اند: (۱) حد ضرر تنگ‌تر از نصف ATR داخل نویز عادی ۵ دقیقه‌ای است و بی‌ربط به درستی تحلیل، خورده می‌شود. (۲) نسبت پاداش/ریسک بالای ۶ تقریباً همیشه نتیجهٔ فروپاشی رگرسور حد ضرر است، نه یک لبهٔ واقعی.

قاعدهٔ جایگزین

```
widen = 1 + 0.6 × garch_vol_rank
if regime == HIGH_VOLATILITY: widen × 1.35
if regime == SIDEWAYS:      widen × 0.85
stop_loss  = atr_pct × 1.0 × widen
take_profit = atr_pct × 2.0 × widen
```

۱۴-۵. مدل ۴: ریسک (RiskModel)

یک رگرسور «امتیاز فرصت» بین ۰ و ۱ می‌دهد. سپس یک منطق کاملاً قطعی (نه یادگیری ماشین) آن را به اهرم و تخصیص سرمایه تبدیل می‌کند.

وتوهای سخت — قبل از هر محاسبه‌ای

- اگر صدک نوسان $0.97 \leq$ ← اهرم ۰، پایان. (بازار خیلی دیوانه است).
- اگر اطمینان جهت $0.70 >$ ← اهرم ۰، پایان.

فرمول اندازه‌گیری

```
confidence_factor = clamp( (confidence - 0.70) / (1 - 0.70), 0, 1 )
volatility_factor = 1 - garch_vol_rank²
tier_factor      = LOW:1.0 MEDIUM:0.75 HIGH:0.5 else:0.6

composite = risk_score × (0.35 + 0.65 × confidence_factor)
           × volatility_factor
           × tier_factor

leverage  = floor(composite × 10)      # اهرم
allocation = 0.01 + (0.20 - 0.01) × composite # تخصیص سرمایه
```

👉 **به زبان بچه‌گانه:** یعنی: هر چه مدل مطمئن‌تر، بازار آرام‌تر، و ریسک مسیر کمتر باشد، هم اهرم بیشتر می‌شود و هم پول بیشتری وارد می‌شود. اگر ضرب این‌ها آن‌قدر کوچک شود که اهرم به ۰ برسد، معامله لغو می‌شود.

نکتهٔ ظریف: اطمینان روی بازهٔ قابل معامله بازمقیاس می‌شود. یعنی سیگنال ۷۰٪ نزدیک کف اندازه می‌گیرد و سیگنال ۱۰۰٪ نزدیک سقف — نه اینکه ۷۰٪ مستقیماً یعنی ۷۰٪ سقف.

قاعدهٔ جایگزین

```
score = 0.4 × direction_confidence
       + 0.3 × (ADX / 50)
       + 0.2 × fdi_trending
       + 0.1 × (1 - garch_vol_rank)
```

۱۴-۶. زیرسیستم مدیریت (MLSubsystem)

کار	مند
-----	-----

بارگذاری هر چهار مدل از دیسک.	load_all()
آموزش هر چهار مدل روی نخ کارگر. شکست یکی، بقیه را متوقف نمی‌کند و در گزارش ثبت می‌شود.	train_all()
اجرای هر چهار مدل روی یک ردیف. تابع خالص و مسدودکننده — بک‌تستر مستقیماً از این استفاده می‌کند.	infer_sync()
نسخه غیرهمزمان، روی نخ کارگر.	infer()
چند ارز موازی، با جداسازی خطای هر ارز.	infer_many()
گزارش کامل: معیارها، ابرداده، و اهمیت ویژگی‌ها برای هر مدل. از آنجا که بارگذاری، ابرداده و مدل را برمی‌گرداند، این گزارش بعد از ری‌استارت هم در دسترس است.	training_report()

ترتیب اجرای پیش‌بینی: جهت ← ورود (که جهت را می‌گیرد) ← خروج ← ریسک (که اطمینان و رده جهت را می‌گیرد). زمان اجرا با ساعت یکنواخت اندازه‌گیری و در latency_ms ثبت می‌شود.

۱۵. قراردادهای بین ماژول‌ها — schemas.py

👤 **به زبان بچه‌گانه:** تصور کن دو کارخانه با هم کار می‌کنند. یکی جعبه می‌سازد و به دیگری می‌دهد. اگر قرار بگذارند «جعبه باید همیشه مربع باشد و کوچک‌تر از ۱ متر»، آن وقت هیچ وقت یک جعبه عجیب به دست کارخانه دوم نمی‌رسد. این فایل همان قرارداد است.

این فایل «کمرباریک» سیستم است: ماژول C فقط از طریق `ModelInferenceResult` با ماژول D حرف می‌زند، و ماژول D فقط از طریق `TradeSignal` با ماژول E. چون هر فیلد اعتبارسنجی و محدود شده، یک کلاس کامل از باگ‌های «مدل مزخرف تولید کرد و ما آن را به صرافی فرستادیم» از نظر ساختاری غیرممکن می‌شود.

۱۵-۱. کلاس‌های خروجی مدل‌ها

DirectionPrediction

- احتمال‌ها اعتبارسنجی می‌شوند: کلاس ناشناخته ممنوع، مقدار منفی ممنوع، و به‌طور خودکار نرمال می‌شوند تا جمعشان دقیقاً ۱ شود.
- خواص محاسبه‌شده: `long_probability`, `short_probability`, `no_trade_probability`, `action` (بزرگ‌ترین)، `confidence`، و `directional_margin` (قدر مطلق فاصله لانگ و شورت).

ExitParameters

- هر سه درصد باید مثبت و $1 \geq$ باشند.
- خاصیت `reward_risk_ratio = TP% / SL%`
- یک اعتبارسنجی: فعال‌سازی تریلینگ نمی‌تواند از حد سود بیشتر باشد (وگرنه هرگز فعال نمی‌شود).

RiskAllocation

- اهرم بین ۰ و ۱۰ — این سقف با اعتبارسنجی قالب اعمال می‌شود، نه فقط با تنظیمات. یعنی حتی اگر کسی تنظیمات را دستکاری کند، اهرم ۲۰ از نظر ساختاری غیرممکن است.
- خاصیت `is_abort`: اگر اهرم ۰ یا تخصیص ۰ باشد.

۱۵-۲. TradeSignal — تنها چیزی که ماژول E اجازه عمل روی آن را دارد

غیرقابل تغییر (frozen). اعتبارسنج هندسه چک می‌کند:

جهت	قوانین
LONG	حد سود باید بالای قیمت ورود باشد، حد ضرر زیر آن، و ماشه تریلینگ بالای آن.
SHORT	دقیقاً برعکس.
NO_TRADE	اصلاً نمی‌تواند سیگنال بسازد — خطا می‌دهد.

❓ **چرا این طوری؟** یک لانگ که حد ضررش بالای قیمت ورود باشد، باگی است که در غیر این صورت فقط با از دست دادن پول کشف می‌شد. اینجا در همان لحظه ساخت گرفته می‌شود.

توابع کمکی محاسبه اندازه:

```
notional(equity) = equity * capital_allocation_pct * leverage
margin(equity)   = equity * capital_allocation_pct
quantity(equity) = notional / reference_price
```

DecisionResult ۱۵-۳

هر تصمیم — چه EXECUTE، چه NO_TRADE، چه BLOCKED — یک شیء کامل تولید می‌کند شامل: شناسهٔ یکتا، نماد، حکم، قانونی که فعال شد، دلیل خوانا، سیگنال (یا None)، نتایج مدل، و لیست کامل checks که هر قانون و نتیجه‌اش را دارد.

۱۶. مرحله ۹: قاضی نهایی — موتور تصمیم و ۱۰ قانون

👤 به زبان بچه‌گانه: مدل‌ها فقط «نظر» می‌دهند. این فایل «تصمیم» می‌گیرد. و هیچ هوش مصنوعی ندارد — فقط ۱۰ سؤال دارد که یکی یکی می‌پرسد. اولین سؤالی که جوابش «نه» باشد، همه چیز تمام می‌شود و دقیقاً همان دلیل در دفتر ثبت می‌شود.

۱۶-۱. آیشار ۱۰ قانون

قانون	شناسه	چه چیزی را چک می‌کند
R0-a	R0_SYSTEM_HALTED	آیا نگهدارنده ریسک فرمز است؟ اگر بله ← BLOCKED.
R0-b	R0_TRADING_DISABLED	آیا اپراتور معامله را مسجل کرده؟
R0-c	R0_PORTFOLIO_FULL	آیا ۵ پوزیشن باز پر شده؟
R0-d	R0_SYMBOL_ALREADY_OPEN	آیا روی همین ارز پوزیشن باز داریم؟
R1-a	R1_DIRECTION_IS_NO_TRADE	آیا خود مدل جهت می‌گوید «معامله نکن»؟
R1-b	R1_DIRECTION_CONFIDENCE_TOO_LOW	احتمال جهت انتخابی ≤ 0.70 است؟
R2	R2_DIRECTIONAL_MARGIN_TOO_THIN	احتمال لانگ - احتمال شورت ≤ 0.15 است؟ (مدل بی تفاوت نباشد).
R3	R3_NO_TRADE_MASS_TOO_HIGH	جرم احتمال «معامله نکن» ≥ 0.35 است؟
R4	R4_ENTRY_MODEL_SAYS_WAIT	مدل زمان بندی می‌گوید «الان»؟
R5	R5_REWARD_RISK_BELOW_FLOOR	نسبت پاداش/ریسک ≤ 1.3 است؟
R6	R6_RISK_MODEL_ABORT	مدل ریسک اهرم غیرصفر داد؟
R7	R7_RISK_TIER_NOT_ACCEPTED	رده ریسک در لیست پذیرفته شده هست؟
R8	R8_REGIME_BLOCKED	رژیم HMM در لیست ممنوعه نیست؟ (پیش فرض: لیست ممنوعه خالی است).
R9	R9_UNTRAINED_MODELS_IN_LIVE_MODE	در حالت واقعی، هیچ مدلی از قاعده جایگزین استفاده نکرده باشد.
R10	R10_SIGNAL_CONSTRUCTION_FAILED	نور ایمنی نهایی: اگر ساخت سیگنال از اعتبارسنجی رد نشد.

❓ چرا این طوری؟ چرا قانون R9 حیاتی است؟ قاعده جایگزین یک حدس مهندسی شده است، نه یک مدل آموزش دیده. اجازه دادن به یک حدس برای ثبت سفارش با پول واقعی و اهرم، دقیقاً همان چیزی است که یک اشکال ساده را به یک فاجعه مالی تبدیل می‌کند. در حالت کاغذی اجازه داده می‌شود (تا بشود سیستم را تست کرد).

۱۶-۲. ساخت سیگنال

وقتی هر ده قانون رد شدند، درصدها به قیمت مطلق تبدیل می‌شوند:

LONG : $TP = price \times (1 + tp\%)$ $SL = price \times (1 - sl\%)$ $trigger = price \times (1 + trail\%)$
 SHORT: $TP = price \times (1 - tp\%)$ $SL = price \times (1 + sl\%)$ $trigger = price \times (1 - trail\%)$

سپس تخصیص سرمایه در `size_multiplier` نگهدارنده ریسک ضرب می‌شود (در حالت زرد ۰.۴). اگر ساخت TradeSignal خطا داد، تابع `None` برمی‌گرداند و قانون R10 فعال می‌شود.

۱۶-۳. ارزیابی دسته‌ای (evaluate_many)

👤 **به زبان بچه‌گانه:** فرض کن ۵ جای خالی برای معامله داری ولی ۱۲ ارز سیگنال داده‌اند. کد اول همه را بر اساس **اطمینان مدل** مرتب می‌کند، بعد از بالا شروع می‌کند. پس ۵ جای خالی به ۵ قوی‌ترین سیگنال می‌رسد، نه به ۵ ارزی که الفبایی اول بودند.

هر بار که یک سیگنال قابل اجرا تولید شود، شمارندهٔ پوزیشن‌های باز برای *ارزیابی ارز بعدی* افزایش می‌یابد. یعنی سقف پرتغوی در همان دسته هم رعایت می‌شود.

۱۶-۴. رد شدن‌ها هم به همان اندازه ثبت می‌شوند

هر قانون که ارزیابی می‌شود، یک ورودی به لیست checks اضافه می‌کند شامل نام قانون، نتیجه (قبول/رد) و جزئیات عددی. مثلاً:

```
{"rule": "R1_DIRECTION_CONFIDENCE_TOO_LOW",  
  "passed": false,  
  "detail": "confidence=0.6800 threshold=0.7000"}
```

همین است که شش هفته بعد از یک افت سرمایه، بتوانی دقیقاً بفهمی چه اتفاقی افتاد.

۱۷. مرحله ۱۰: اجرای معامله — کاغذی و واقعی

👤 **به زبان بچه‌گانه:** دو «پیشخدمت» داریم که دقیقاً یک شکل هستند: یکی غذا را واقعاً می‌برد (LiveExecutor)، یکی فقط وانمود می‌کند (PaperTrader). چون شکلشان یکی است، حلقه معاملاتی حتی یک خط `if paper:` ندارد.

۱۷-۱. ساختار مشترک: `Position (models.py)`

همین یک کلاس در اجرای واقعی، کاغذی و بک‌تست استفاده می‌شود. این عمده است: ریاضیات یکسان سود/زیان، کارمزد، فاندینگ و تریلینگ در هر سه موتور، همان چیزی است که نتیجه بک‌تست را با نتیجه واقعی قابل مقایسه می‌کند.

محاسبات کلیدی

```
direction      = +1 (LONG) / -1 (SHORT)
unrealized_pnl = (mark_price - entry_price) * quantity * direction
notional       = quantity * entry_price
effective_stop() = trailing_stop if trailing_active else stop_loss
```

قیمت لیکوئید (تقریب مارجین ایزوله)

```
LONG : entry * (1 - 1/leverage + mmr)
SHORT: entry * (1 + 1/leverage - mmr)      mmr = 0.005
```

👤 **به زبان بچه‌گانه:** با اهرم ۱۰، قیمت لیکوئید لانگ حدود $\text{entry} \times (1 - 0.1 + 0.005)$ یعنی حدود ۹۰۵٪ زیر قیمت ورود است. اگر قیمت آنجا برسد، کل پول ودیعه از بین می‌رود.

تریلینگ استاپ (`advance_trailing`)

- تا وقتی قیمت به «ماشه تریلینگ» نرسیده، تریلینگ فعال نمی‌شود.
- بعد از فعال شدن، حد ضرر فقط در جهت سود حرکت می‌کند (جفجه‌ای — `ratchet`).

❓ **چرا این‌طوری؟** یک تریلینگ که بتواند شل شود، یک باگ است که بی‌سروصدا ریسک را زیاد می‌کند. کد صراحتاً چک می‌کند که سطح جدید بهتر از سطح قبلی باشد، وگرنه `None` برمی‌گرداند و هیچ تغییری نمی‌دهد.

۱۷-۲. موتور اجرای واقعی (`executor.py`)

فرض عملیاتی: بازار و API دشمن‌اند.

گام‌های باز کردن یک پوزیشن

۱. بررسی نگرهان ریسک. اگر قرمز است `KillSwitchEngaged`. اگر سقف روزانه معاملات پر شده ← رد.

۲. بررسی پوزیشن تکراری. اگر روی این ارز پوزیشن باز است ← رد.

۳. گرفتن وضعیت حساب از صرافی (موجودی USDT + سود/زیان باز).

۴. محاسبه و اعتبارسنجی مقدار (`_resolve_quantity`): مقدار خام محاسبه، سپس با `amount_to_precision` گرد

می‌شود، سپس در برابر حداقل مقدار و حداقل ارزش صرافی چک می‌شود، و آخر اینکه مارجین لازم نباید از سرمایه بیشتر باشد.

۵ پیش‌پرواز (preflight). حالت مارجین ISOLATED و اهرم همین سیگنال صراحتاً روی صرافی تنظیم می‌شوند.

? چرا این‌طوری؟ چرا هر بار؟ چون اگر جلسه قبلی اهرم را روی ۲۰ گذاشته باشد و ما تنظیم نکنیم، اندازه پوزیشن بی‌سروصدا دو برابر می‌شود. کد پیام‌های بی‌ضرر بایننس («نیازی به تغییر نیست») را می‌بلعد.

۶ ارسال سفارش ورود. پیش‌فرض سفارش بازار. اگر سفارش محدود انتخاب شده باشد، تا ۴۵ ثانیه منتظر پر شدن می‌ماند، بعد لغو می‌کند. اگر بخشی پر شده باشد، همان بخش نگه داشته می‌شود.

? چرا این‌طوری؟ چرا باقی‌مانده را با سفارش بازار دنبال نمی‌کند؟ چون تعقیب قیمت با یک قیمت بدتر، دقیقاً همان‌طور است که یک لبه کوچک به لبه منفی تبدیل می‌شود.

۷ ثبت سفارش‌های محافظ. دو سفارش `reduceOnly: TAKE_PROFIT_MARKET` و `STOP_MARKET`، هر دو با `workingType=MARK_PRICE`.

? چرا این‌طوری؟ چرا قیمت مارک و نه آخرین قیمت؟ چون خود بایننس لیکوئیدیشن را با قیمت مارک ارزیابی می‌کند. اگر حد ضرر ما با «آخرین قیمت» فعال شود، یک فیتیل لحظه‌ای که هرگز به مارک نرسیده می‌تواند ما را بیرون بیندازد.

۸

⚠ هشدار: اگر ثبت سفارش محافظ شکست بخورد، پوزیشن فوراً با سفارش بازار بسته می‌شود. یک پوزیشن اهرمی بدون حد ضرر، بدترین حالت ممکن است — بدتر از اینکه اصلاً معامله نکنی.

۹ ثبت پوزیشن در حافظه و در جدول `trades`.

حلقه نظارت (هر ۵ ثانیه)

- قیمت همه پوزیشن‌ها به‌صورت دسته‌ای گرفته می‌شود.
- بیشینه/کمینه مسیر به‌روزرسانی می‌شود.
- تریلینگ در صورت لزوم جلو می‌رود.
- هر ۶۰ ثانیه یک تطبیق با صرافی انجام می‌شود.

جابه‌جایی تریلینگ روی صرافی

بایننس تریلینگ با هندسه دلخواه ما را ندارد، پس سفارش قدیمی لغو و سفارش جدید ثبت می‌شود. ترتیب مهم است: **اول لغو، بعد ثبت**.

? چرا این‌طوری؟ چون یک لحظه بدون حد ضرر بودن، امن‌تر از یک لحظه دو حد ضرر داشتن است — دو تا حد ضرر یعنی پوزیشن دوبار بسته می‌شود و ناگهان یک پوزیشن معکوس باز می‌کنی! اگر ثبت مجدد شکست بخورد، کد پوزیشن را فوراً می‌بندد.

تطبیق (reconcile)

لیست پوزیشن‌های واقعی صرافی خوانده می‌شود. هر چیزی که ما ردیابی می‌کنیم ولی صرافی دیگر گزارش نمی‌دهد، یعنی بسته شده (با حد سود، حد ضرر یا لیکوئیدیشن). کد سفارش‌های TP و SL را چک می‌کند تا بفهمد کد/میک پر شده و دلیل بسته شدن را درست ثبت کند. پوزیشن‌های ناشناخته صرافی فقط هشدار می‌گیرند (کد آن‌ها را مدیریت نمی‌کند).

۱۷-۳. موتور کاغذی (paper_trader.py)

رابط دقیقاً یکسان، ولی وانمود نمی‌کند که معامله شبیه‌سازی شده رایگان است:

هزینه	نحوه اعمال
کارمزد	ورود و خروج هر دو با نرخ تیکر ۰.۰۵٪. کارمزد ورود بلافاصله از موجودی کم می‌شود، دقیقاً مثل صرافی.
لغزش	۵ پایه (۰.۰۵٪) همیشه به ضرر ما. ورود لانگ و خروج شورت هر دو «خرید» هستند، پس هر دو به بالا جریمه می‌شوند.
فاندینگ	هر ۸ ساعت، با نرخ فاندینگ که واقعاً مازول A برای همان ارز ثبت کرده.
لیکوئیدیشن	قبل از حد سود چک می‌شود. هیچ تخفیف لغزشی هم داده نمی‌شود — پوزیشن دقیقاً در قیمت لیکوئید پاک می‌شود و ضرر به اندازه مارجین محدود می‌شود.

ترتیب بررسی موانع — عمداً بدبینانه

- ببند، کل مارجین رفت --> بله --> لیکوئیدیشن ۱.
- ببند با ضرر --> بله --> (یا تریلینگ) حد ضرر ۲.
- ببند با سود --> بله --> حد سود ۳.

؟ چرا این طوری؟ یک «تیک» قیمتی که می‌توانست چند مانع را همزمان بزند، **همیشه** بدترین نتیجه را ثبت می‌کند. هر ترتیب دیگری، نتیجه شبیه‌سازی را از واقعیت خوش‌بینانه‌تر می‌کند.

موجودی مجازی در کلید paper_trader_state پایگاه داده ذخیره می‌شود، پس بعد از ری‌استارت از همان‌جا ادامه می‌دهد.

۱۸. مرحله ۱۱: نگهبان ریسک — risk_guard.py

👤 **به زبان بچه‌گانه:** همهٔ بخش‌های دیگر اجازه دارند اشتباه کنند. این یکی با این فرض ساخته شده که **حتماً** اشتباه خواهند کرد. مدل‌ها منحرف می‌شوند، صرافای قطع می‌شود، و یک زنجیرهٔ باخت دقیقاً شبیه یک استراتژی موفق است — تا لحظه‌ای که حساب خالی شود.

۱۸-۱. ماشین حالت سهرنگ

حالت	معنی	اثر
GREEN	عادی	اندازهٔ کامل معامله (ضریب ۱٫۰).
YELLOW	هشدار	اندازهٔ معامله ۰٫۴× می‌شود. معامله ادامه دارد ولی محتاطانه.
RED	توقف کامل	همهٔ سفارش‌ها لغو، همهٔ پوزیشن‌ها بسته، اجرا غیرفعال، و قفل می‌شود تا آدم دستی بازش کند.

۱۸-۲. شرایط رفتن به قرمز

شرط	آستانه
افت سرمایه از ابتدای روز	$\leq 5\%$
افت از اوج تاریخی سرمایه	$\leq 20\%$
باخت پی‌پی	۵ معامله
خطای صرافای در پنجرهٔ ۵ دقیقه‌ای	۶ خطا
فشار دادن کلید قرمز در پنل	فوری

۱۸-۳. شرایط زرد

- افت روزانه $\leq 3\%$ ، یا
- ۳ باخت پی‌پی، یا
- ۳ خطای صرافای در پنجره.

زرد **برگشت‌پذیر** است: وقتی شرایط هشدار برطرف شد، خودکار به سبز برمی‌گردد.

۱۸-۴. پنجرهٔ لغزان خطاها

خطاهای API با زمانشان در یک صف نگه داشته می‌شوند و فقط خطاهای داخل پنجرهٔ ۳۰۰ ثانیه‌ای شمرده می‌شوند.

❓ **چرا این‌طوری؟** یک قطعی موقت که خودش رفع شده، نباید تا ابد یک زخم روی سیستم بگذارد. ولی یک API واقعاً خراب — دقیقاً وضعیتی که ثبت کورکورانهٔ سفارش خطرناک‌ترین است — باید سیستم را متوقف کند. یک تماس موفق، پنجره را پاک می‌کند.

۱۸-۵. قفل ماندگار (Latch)

⚠️ **هشدار:** وقتی نگهبان قرمز می‌شود، وضعیتش در **پایگاه داده نوشته می‌شود**. هنگام راه‌اندازی بعدی، متد `load()` آن را می‌خواند و دوباره قرمز می‌شود. یعنی یک حلقهٔ کرش (برنامه بیفتد و دوباره بالا بیاید) نمی‌تواند بی‌سروصدا معامله را دوباره فعال کند. باز کردن قفل فقط با `POST /api/reset_risk_guard` یا دکمهٔ پنل ممکن است.

هنگام باز کردن قفل: حالت به سبز، دلیل پاک، زنجیره باخت صفر، پنجره خطاها خالی، و سرمایه شروع روز روی سرمایه فعلی تنظیم می‌شود. معامله همچنان مسلح نمی‌شود — باید جداگانه مسلح شود.

۱۸-۶. الگوی فراخوان (Callback)

موتور اجرا تابع `emergency_flatten` خودش را در نگه‌بان ثبت می‌کند. وقتی نگه‌بان قرمز می‌شود، همه فراخوان‌های ثبت‌شده را اجرا می‌کند.

چرا این‌طوری؟ چرا فراخوان و نه وابستگی مستقیم؟ چون این‌طوری نگه‌بان ریسک هیچ اطلاعی از جزئیات داخلی مازول E ندارد. همچنین اگر یک فراخوان خطا بدهد، فقط لاگ می‌شود: یک روال تخلیه خراب نباید جلوی خود توقف را بگیرد.

۱۸-۷. غلتش روز UTC

در ساعت ۰۰:۰۰ UTC، شمارنده‌های روزانه صفر می‌شوند: تعداد معاملات، بردها، باخت‌ها، سود/زیان محقق‌شده، و سرمایه شروع روز. اگر برنامه در طول شب خاموش بوده، این کار هنگام بارگذاری انجام می‌شود.

۱۹. مرحله ۱۲: دفتر خطرات سیستم — audit_engine.py

👤 **به زبان بچه‌گانه:** شش هفته بعد از یک ضرر، جمله «مدل گفت» هیچ کمکی نمی‌کند. ولی این جمله کمک می‌کند: «در ۱۴ مارس ساعت ۰۹:۱۵ به وقت UTC، بیت‌کوین رد شد چون اطمینان جهت ۰.۶۸ در برابر آستانه ۰.۷۰ بود، با HMM در رژیم ۲ و نوسان GARCH در صدک ۹۳.»

۱۹-۱. چه چیزی ثبت می‌شود

هر تصمیم — از جمله هر **NO_TRADE**. چون جالب‌ترین شکست‌ها دقیقاً همان‌جا مخفی‌اند. هر رکورد بیش از ۳۰ ستون دارد:

گروه	ستون‌ها
شناسه	شناسه تصمیم، شناسه چرخه (یکسان برای همه ارزهای یک چرخه)، نماد، زمان کندل، زمان ایجاد.
حکم	حکم، اقدام، قانون فعال‌شده، دلیل خوانا.
تصویر ویژگی	رژیم HMM، نوسان GARCH، صدک نوسان، شیب ATR، FDI، KAMA، عدم توازن دفتر سفارش، قیمت بسته.
خروجی مدل‌ها	احتمال لانگ/شورت/عدم معامله، اطمینان، احتمال ورود، اجازه ورود.
هندسه	درصد حد سود/ضرر/تریلینگ، اهرم پیشنهادی، درصد تخصیص سرمایه، رده ریسک.
سیستم	حالت نگهدار ریسک، زمان پاسخ به میلی‌ثانیه.
محتوای JSON	توزیع کامل احتمال‌ها، منبع هر مدل (آموزش‌دیده یا جایگزین)، نسبت پاداش/ریسک، دلیل لغو ریسک، نسخه مدل‌ها، و لیست کامل هر ۱۰ قانون با نتیجه‌شان.

۱۹-۲. نوشتن بدون بلوکه کردن

```
حلقه معاملاتی
|
| put_nowait() (بدون انتظار)
v
(حداکثر ۴۰۰۰ رکورد) asyncio صف
|
| تسک پس‌زمینه، هر ۲ ثانیه
v
SQLite یک تراکنش --> دسته‌های ۶۴ تایی
```

❓ **چرا این طوری؟** از دست دادن یک خط از دفتر بد است. ولی متوقف کردن حلقه معاملاتی پشت دیسک بدتر است. پس وقتی صف پر می‌شود، قدیمی‌ترین رکورد دور ریخته و شمرده می‌شود، و هر ۱۰۰ رکورد از دست‌رفته یک هشدار در لاگ می‌آید.

هنگام خاموشی، stop(drain=True) صف را کاملاً روی دیسک خالی می‌کند. همچنین قبل از خواندن رکوردها برای پل، یک flush() اجباری انجام می‌شود تا تازه‌ترین تصمیم‌ها هم دیده شوند.

۱۹-۳. رویدادهای سیستمی

غیر از تصمیم‌های هر ارز، رویدادهای کلی هم ثبت می‌شوند با نماد *:

قانون	رویداد
OPERATOR_ARM	اپراتور معامله را مسلح کرد.
OPERATOR_DISARM	اپراتور معامله را متوقف کرد.

لیست ارزها ذخیره شد.	OPERATOR_UNIVERSE
کلید قرمز فشرده شد.	OPERATOR_KILL_SWITCH
قفل نگهبان باز شد.	OPERATOR_RESET
هیچ ارزی از کنترل کیفیت رد نشد.	QC_ALL_FAILED
هیچ ارزی ردیف ویژگی گرم تولید نکرد.	FEATURES_COLD
چرخه با خطا مواجه شد.	CYCLE_FAILURE
خط لوله راه اندازی شکست خورد.	SETUP_FAILURE

۲۰. مرحله ۱۳: پنل وب — web_app.py و templates.py

👤 به زبان بچه‌گانه: پنل، «اتاق کنترل» است. با مرورگر به آدرس آی‌پی‌سرور: http://8000 می‌روی و همه‌چیز را می‌بینی و دکمه فشار می‌دهی. هیچ دامنه، هیچ گواهی SSL، و هیچ پروکسی لازم نیست — که دقیقاً نکته مثبت برای یک سرور کوچک است.

۲۰-۱. چهار صفحه HTML

آدرس	محتوا
/	داشبورد: نوار پیشرفت راه‌اندازی، ۷ کارت وضعیت (فاز، نگرهان ریسک، حالت، سرمایه، افت روزانه، پوزیشن‌های باز، آخرین چرخه)، دکمه‌های کنترل، جدول پوزیشن‌های فعال، سلامت مدل‌ها، گزارش آموزش ML، آخرین تصمیم‌ها، و لاگ زنده.
/universe	انتخاب ارز: جدول کامل ارزهای بایننس با قیمت، حجم ۲۴ ساعته، تغییر ۲۴ ساعته، اسپرد، حداقل سفارش، هزینه پله مقدار، حداکثر اهرم، روزهای لیست‌شدن، امتیاز، و وضعیت. با چک‌باکس، فیلتر و دکمه «پیشنهاد».
/audit	دفتر تصمیم‌ها: ۲۲ ستون شامل همه احتمالات و قانون فعال‌شده، با فیلتر بر اساس نماد و حکم. هر ۱۵ ثانیه تازه می‌شود.
/trades	تاریخچه معاملات: باز شدن، بسته شدن، اهرم، مقدار، ورود، خروج، کارمزد، فاندینگ، سود/زیان، و دلیل بسته شدن.

۲۰-۲. نقاط API

فقط خواندنی (بدون رمز)

مسیر	خروجی
GET /health	بررسی زنده‌بودن پروسه.
GET /api/status	وضعیت کامل: فاز، پیشرفت، حالت، نگرهان ریسک، حساب، پوزیشن‌ها، آمار آخرین چرخه، و سلامت اجزا.
GET /api/ml/report	گزارش آموزش هر چهار مدل + اهمیت ویژگی‌ها + پوشش داده. قابل داندلود به‌صورت JSON.
GET /api/audit	رکوردهای دفتر با فیلتر.
GET /api/trades	معاملات.
GET /api/positions	پوزیشن‌های باز.
GET /api/logs	آخرین خطوط لاگ از حافظه.
GET /api/universe/available	لیست غربال‌شده ارزها.
GET /api/universe/suggest	بهترین ارزهای واجد شرایط.
GET /api/universe/selected	انتخاب فعلی.
GET /api/setup/status	پیشرفت جمع‌آوری و آموزش.

تغییردهنده (نیازمند رمز، اگر تنظیم شده باشد)

مسیر	کار
POST /api/universe/select	ذخیره انتخاب ارزها و شروع خودکار راه‌اندازی.
POST /api/setup/start	اجرای دوباره جمع‌آوری؛ با {"force_retrain": true} آموزش اجباری.
POST /api/trading/start	مسلح کردن با {"mode": "paper" "live"}.
POST /api/trading/stop	خلع سلاح. با {"flatten": true} پوزیشن‌ها هم بسته می‌شوند.
POST /api/toggle_trading	مکث/ادامه و/یا تعویض حالت کاغذی↔واقعی.

POST /api/kill_switch	قرمز کردن فوری و بستن همه چیز.
POST /api/reset_risk_guard	باز کردن قفل قرمز.

۳-۲۰. سه جزئیات مهندسی جالب

الف) SafeJSONResponse — پنل هرگز سفید نمی‌شود

چرا این طوری؟ داشبورد هر چند ثانیه `/api/status` را می‌خواند. اگر یک عدد غیرممتناهی (NaN یا بی‌نهایت) در پاسخ باشد — که از یک معیار آموزش خالی یا یک تقسیم در حالت تنزل یافته به راحتی تولید می‌شود — کتابخانه `Starlette` هنگام رندر خطا می‌دهد و **کل داشبورد سفید می‌شود**. این کلاس همه چیز را بازگشتی تمیز می‌کند: عدد غیرممتناهی ← `null`، اعداد `NumPy` ← عدد پایتون، `Decimal` و تاریخ ← رشته. (کامیت 05e6504)

ب) Protocol به جای `import` مستقیم

کلاس `SystemController` یک `Protocol` است که فقط می‌گوید «هر چیزی که این متدها را داشته باشد». این طوری ماژول `F` هیچ وقت `main.py` را `import` نمی‌کند و وابستگی حلقوی بین پنل و ارکستر ایجاد نمی‌شود.

ج) قالب‌ها داخل خود پایتون

همه `HTML` و `JavaScript` در دیکشنری `TEMPLATES` در فایل `templates.py` است. نه پوشه `/static`، نه `node_modules`، نه مرحله `build`. `Tailwind` از `CDN` می‌آید ولی یک `CSS` درون خطی فشرده هم هست تا اگر سرور دسترسی اینترنت خروجی نداشت، صفحه باز هم کاملاً خوانا بماند.

یک باگ واقعی که رفع شد: کاراکترهای `backslash` در رشته‌های `JavaScript` هنگام تبدیل به رشته پایتون بلعیده می‌شدند و باعث `SyntaxError` و سفید شدن صفحه می‌شدند. (کامیت 311e136)

۴-۲۰. وضعیت امنیتی

⚠ هشدار: پنل برای اینترنت عمومی سخت‌سازی نشده است. اگر `WEB_API_TOKEN` را تنظیم کنی، همه نقاط تغییردهنده هدر `X-API-Token` می‌خواهند، ولی نقاط فقط-خواندنی باز می‌مانند. پورت را حتماً پشت فایروال، لیست سفید `IP`، یا تونل `SSH` بگذار.

۵-۲۰. فازهای سیستم (`setup_state.py`)

فار	معنی
STARTING	در حال بالا آمدن.
AWAITING_UNIVERSE	منتظر انتخاب ارز توسط اپراتور.
COLLECTING_DATA	در حال دانلود تاریخچه.
TRAINING	در حال آموزش مدل‌ها.
READY	آماده — ولی هنوز معامله نمی‌کند.
PAPER_TRADING	مسلح در حالت کاغذی.
LIVE_TRADING	مسلح با پول واقعی.
SETUP_FAILED	راه‌اندازی شکست خورد. پایانی است تا اپراتور دوباره تلاش کند — هرگز بی‌سروصدا با مدل‌های کهنه شروع به معامله نمی‌کند.

کلاس `SetupProgress` نوار پیشرفت را نگه می‌دارد: مرحله، جزئیات، درصد، تعداد ارز انجام‌شده/کل، زمان شروع/پایان، خطا، خلاصه داده، و خلاصه آموزش.

۲۱. بک تست — امتحان روی گذشته (backtester.py)

👤 به زبان بچه‌گانه: بک تست یعنی «فیلم بازار را از اول پخش کن و بین ربات چه می‌کرد». ولی خیلی راحت می‌شود در بک تست تقلب کرد — و آن وقت نتیجه درخشانی می‌گیری که در واقعیت تکرار نمی‌شود. این فایل عمداً طوری نوشته شده که تو را ناامید کند.

۲۱-۱. پنج قانون واقع‌گرایی

۱. پر شدن در «باز شدن کندل بعدی»، نه بسته شدن کندل فعلی.

? چرا این‌طوری؟ تصمیمی که در لحظه بسته شدن کندل t گرفته شده، غیرممکن است که داخل همان کندل اجرا شده باشد. پر کردن با `close[t]` رایج‌ترین راه ساختن سودهای خیالی است.

۲. بدترین ترتیب داخل کندل. برای یک لانگ، بالاتر حد ضرر و قیمت لیکوئید در راه پایین اول لمس می‌شود، پس همان سطح برنده است. فقط اگر هیچ‌کدام از سطوح نامطلوب لمس نشده باشند، حد سود بررسی می‌شود.

۳. مدل هزینه کامل. کارمزد تیکر روی هر دو پا، لغزش روی هر پر شدن، و فاندینگ در مرزهای واقعی `08:00` و `16:00` UTC با نرخ‌هایی که مازول A واقعاً ثبت کرده.

۴. لیکوئیدیشن مارجین ایزوله. در هر کندل چک می‌شود و یک پوزیشن لیکوئید شده کل مارجینش را از دست می‌دهد، نه بیشتر و نه کمتر.

۵. محدودیت‌های سطح پرتفوی. همان سقف ۵ پوزیشن همزمان و یک پوزیشن در هر ارز که موتور زنده اعمال می‌کند.

۲۱-۲. حلقه شبیه‌سازی

برای هر کندل روی خط زمان مشترک، چهار گام:

۱. حل موانع پوزیشن‌های باز با این کندل
(جلو بردن تریلینگ >-- حد سود >-- حد ضرر >-- لیکوئید >-- فاندینگ)

۲. تولید کرده بود * قبلی * پر کردن سیگنال‌هایی که کندل
این کندل، با لغزش * باز شدن * در قیمت
(سیگنال فقط یک کندل اعتبار دارد؛ اگر نشد، دور ریخته می‌شود)

۳. این کندل * بسته شدن * تولید سیگنال جدید از قیمت

۴. ارزشگذاری به بازار، ثبت نقطه منحنی سرمایه

یک نکته ظریف: بعد از فعال شدن تریلینگ روی سقف کندل، دوباره کف همان کندل در برابر حد ضرر تنگ شده تست می‌شود. کندی که بالا رفت و هم‌اکنون را پس داد، نباید تا کندل بعدی فرار کند.

۲۱-۳. معیارهای خروجی

معیار	معنی
تعداد معاملات، نرخ برد	پایه‌ای‌ترین آمار.

ضرب سود (Profit Factor)	سود ناخالص ÷ زیان ناخالص. بالای ۱ یعنی سودده.
امید ریاضی (Expectancy)	میانگین سود/زیان هر معامله به USDT.
حداکثر افت سرمایه	بدترین ریزش از اوج تا کف.
نسبت شارپ	بازدهی تقسیم بر نوسان، سالانه شده با ۱۰۵'۱۲۰ کندل در سال.
نسبت سورتینو	مثل شارپ ولی فقط نوسان منفی را جریمه می کند.
نسبت کالمار	بازدهی سالانه ÷ حداکثر افت.
کارمزد و فاندینگ کل	هزینه واقعی معامله گری.
تعداد لیکوئیدیشن	چند بار حساب زده شد.

۲۱-۴. هشدار مهم درباره نتایج

⚠ **هشدار:** فایل README پروژه با صداقت این عددها را می آورد:

سناریو	معاملات	نرخ برد	ضرب سود	بازدهی
درون نمونه (روی همان داده آموزش دیده)	۲۷	۹۲٫۶٪	۳۱٫۶	+۰٫۵۷٪
خارج نمونه (داده ندیده)	۵	۰٪	۰٫۰	-۰٫۰۶٪

همین شکاف، کل نکته است. اجرای backtest بلافاصله بعد از train بی معنی است، چون مدل ها همان داده را دیده اند. قبل از باور کردن هر عددی، یک بازه زمانی که مدل ها هرگز لمسش نکرده اند را کنار بگذار و روی آن تست کن.

۲۲. یک چرخه ۵ دقیقه‌ای کامل، ثانیه‌به‌ثانیه

فرض کنیم ساعت UTC 14:35:10 است و سیستم در حالت کاغذی مسلح است.

مرحله	جزء	چه اتفاقی می‌افتد
۰	زمان‌بند	APScheduler تابع trading_cycle() را صدا می‌زند. اگر چرخه قبلی هنوز در حال اجراست، این نوبت رد می‌شود.
۱	قفل چرخه	یک شناسه یکتا (UUID) برای چرخه ساخته می‌شود و در همه رکوردها استفاده می‌شود.
۲	ماژول A	برای هر ۳۰ ارز موازی: گرفتن کندل‌ها + دفتر سفارش + متریک‌های آتی؛ عبور از QC؛ تعمیر در صورت نیاز؛ ذخیره در دیتابیس. ارزهای رد شده حذف می‌شوند.
۳	دروازه معامله	اگر هیچ ارزی از QC رد نشد ← رویداد QC_ALL_FAILED و پایان. اگر معامله مسلح نیست ← اینجا تمام می‌شود. داده تازه ماند ولی ماژول‌های B تا F اجرا نمی‌شوند.
۴	حساب + ریسک	وضعیت حساب گرفته می‌شود و به نگهدارنده ریسک داده می‌شود. نگهدارنده حالت (سبز/زرد/قرمز) را برمی‌گرداند.
۵	ماژول B	برای هر ارز باقی‌مانده: خواندن حدود ۱۲۲۰ کندل از دیتابیس، ساخت ۲۶ ویژگی، و استخراج فقط آخرین ردیف کامل . اگر آن ردیف کهنه بود، ارز رد می‌شود.
۶	ماژول C	هر چهار مدل روی هر ردیف اجرا می‌شوند (موازی، روی نخ‌های کارگر). خروجی: یک ModelInferenceResult برای هر ارز.
۷	ماژول D	مرتب‌سازی بر اساس اطمینان، سپس آنبشار ۱۰ قانون برای هر ارز. شمارنده پوزیشن‌ها در طول دسته به‌روز می‌شود.
۸	ماژول E	برای هر تصمیم EXECUTE: باز کردن پوزیشن. خطاها ایزوله می‌شوند و کل چرخه را نمی‌کشند.
۹	ماژول F	هر تصمیم — اجرا شده یا نشده — در صف دفتر ثبت می‌شود، همراه با جزئیات اجرا در صورت وجود.
۱۰	پایان چرخه	یک نقطه به منحنی سرمایه اضافه می‌شود، خلاصه لاگ می‌شود، و مدت‌زمان چرخه ثبت می‌شود.

نکته: یک تصمیم طراحی مهم: ماژول A همیشه اجرا می‌شود، حتی وقتی معامله خلع سلاح است. چرا؟ چون این‌طوری تاریخچه کندل‌ها همیشه گرم می‌ماند و وقتی تو دکمه «شروع» را می‌زنی، از همان کندل بعدی کار می‌کند — نه اینکه منتظر بمانی ویژگی‌های کند دوباره گرم شوند.

۲۲-۱. به‌طور موازی، در پس‌زمینه

تسک	تناوب
نظارت بر پوزیشن‌ها	هر ۵ ثانیه: قیمت‌ها، تریلینگ، بررسی موانع.
تطبیق با صرافی (فقط زنده)	هر ۶۰ ثانیه.
خالی کردن صف دفتر	هر ۲ ثانیه.
سرور وب	همیشه فعال، پاسخ به درخواست‌های مرورگر.

۲۳. چرخه عمر سیستم و فازها

```
python main.py
|
v
[STARTING] --- پنل وب فوراً بالا می‌آید ---
|
| آیا ارزی ذخیره شده؟ +-+
|
| بله نه
|
v
[AWAITING_UNIVERSE]
|
| ارز تیک میزنی /universe تو در
| می‌کنی SAVE و
|
|
+-----> [COLLECTING_DATA] <----+
|
| (بکفیل تا ۲ سال کندل)
|
v
[TRAINING]
|
| (۴ مدل روی دیتاست ادغام‌شده)
|
v
[READY] <---- هنوز هیچ معامله‌ای نمی‌کند <----
|
| را میزنی GO LIVE یا START PAPER تو دکمه
|
+-----+-----+
|
v
[PAPER_TRADING] <-----> [LIVE_TRADING]
|
+----- STOP TRADING -----+
|
v
[READY]

(پایانی تا تلاش مجدد) [SETUP_FAILED] --> هر جای این مسیر، خطا
همه چیز بسته و قفل --> RED نگهدار --> هر جای این مسیر، کلید قرمز
```

۲۳-۱. سه ضامن ایمنی در این چرخه

1. هیچ وقت خودکار شروع نمی‌شود. فاز READY یک ایست کامل است. مگر اینکه صریحاً `AUTOSTART_PAPER_TRADING=true` گذاشته باشی.
2. حالت واقعی چهار مدل آموزش‌دیده می‌خواهد. اگر حتی یکی از آن‌ها قاعده جایگزین باشد، مسلح شدن رد می‌شود.
3. شکست راه‌اندازی پایانی است. سیستم هرگز بی‌سروصدا با مدل‌های کهنه معامله را ادامه نمی‌دهد.

۲۴. دستورهای خط فرمان

دستور	کار
python main.py python main.py run	مسیر اصلی: پنل + راه‌اندازی خودکار + زمان‌بند ۵ دقیقه‌ای.
python main.py universe	چاپ جدول غربال‌شدهٔ ارزها در ترمینال (بدون نیاز به پنل). ستون‌ها: نماد، قیمت، حجم ۲۴ ساعته، اسپرد، حداقل سفارش، هزینهٔ پله، روزهای لیست‌شدن، و وضعیت واجد شرایط بودن.
python main.py bootstrap	فقط دانلود تاریخچهٔ کندل‌ها.
python main.py train	ساخت دیتاست و آموزش هر چهار مدل.
python main.py backtest --candles 5000 --equity 1000	پخش مجدد تاریخچه با معیارهای کامل.
python main.py cycle	اجرای دقیقاً یک چرخه سپس خروج (مفید برای cron یا اشکال‌زدایی).
--mode paper live	بازنویسی حالت اجرا.

دستورهای تک‌شات از ارزهای ذخیره‌شده در پنل استفاده می‌کنند. اگر هیچ‌چیز ذخیره نشده باشد، خودکار ارزهای برتر غربال‌شده را انتخاب و ذخیره می‌کنند (_resolve_cli_universe) تا CLI به‌تنهایی قابل استفاده باشد.

۲۴-۱. راه‌اندازی از صفر

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env          # اختیاری
python main.py
```

سپس در مرورگر: <http://8000>/آی‌پی‌سرور

۲۵. جدول‌های پایگاه داده — نمای کامل

ohlcv

id, symbol, timeframe, timestamp, open, high, low, close, volume, inserted_at
کلید یکتا: (symbol, timeframe, timestamp) • ایندکس: (symbol, timestamp)

order_book_snapshots

id, symbol, timestamp, best_bid, best_ask, spread, spread_bps, bid_volume, ask_volume, imbalance, microprice, levels, inserted_at

futures_metrics

id, symbol, timestamp, funding_rate, next_funding_time, open_interest, open_interest_value, long_short_ratio, top_trader_long_short_ratio, taker_buy_sell_ratio, liquidation_buy_volume, liquidation_sell_volume, mark_price, index_price, inserted_at

audit_logs

id, decision_id, cycle_id, symbol, candle_timestamp, created_at, verdict, action, rule_triggered, reason, hmm_regime, garch_volatility, garch_vol_percentile, kama_slope, fdi, atr, order_book_imbalance, close_price, prob_long, prob_short, prob_no_trade, direction_confidence, entry_probability, entry_allowed, take_profit_pct, stop_loss_pct, trailing_trigger_pct, recommended_leverage, capital_allocation_pct, risk_tier, risk_guard_state, latency_ms, payload(JSON)

trades

id, decision_id (یکتا), symbol, mode, side, status, leverage, quantity, notional, margin, entry_price, exit_price, take_profit, stop_loss, trailing_trigger, trailing_stop, liquidation_price, fees_paid, funding_paid, realized_pnl, max_adverse_excursion, max_favorable_excursion, opened_at, closed_at, close_reason, exchange_order_id, payload(JSON)

equity_curve

id, mode, timestamp, balance, equity, unrealized_pnl, open_positions, drawdown_pct, created_at

system_state — کلید/مقدار JSON

کلید	محتوا
universe_selection	لیست ارزش‌های انتخابی، زمان، و نام اپراتور.
trained_universe	لیست ارزش‌هایی که مدل‌ها روی آن‌ها آموزش دیده‌اند + زمان + تعداد ردیف.
risk_guard_state	حالت، دلیل توقف، سرمایه شروع روز، اوج سرمایه، زنجیره باخت، معاملات امروز.
paper_trader_state	موجودی مجازی.

۲۶. لایه‌های ایمنی – کجا جلوی فاجعه گرفته می‌شود

یک نمای کلی از همه نقاطی که کد عمداً «نه» می‌گوید. این‌ها را به ترتیب از داده تا پول مرتب کرده‌ام.

لایه	محل	چه چیزی را متوقف می‌کند
۱	models.py (Pydantic)	کندلی که $low < high$ دارد یا قیمت منفی دارد، اصلاً ساخته نمی‌شود.
۲	fetcher.py	ردیف خراب حذف می‌شود؛ کندل نیمه‌تمام حذف می‌شود؛ دفتر متقاطع None برمی‌گردد؛ فاندینگ غیرمنطقی صفر می‌شود.
۳	qc_validator.py	کندل گمشده، تکراری، ناهم‌تراز، از آینده، کهنه، جهش قیمت، فید مرده. ارز تعمیرناپذیر برای آن چرخه کاملاً حذف می‌شود.
۴	features.py	هیچ ویژگی‌ای اجازه نگاه به آینده ندارد. برآزش‌های ناموفق GARCH/HMM به روش‌های ساده‌تر تنزل می‌کنند، نه به NaN.
۵	processor.py	ردیف‌های گرم‌نشده حذف می‌شوند؛ ردیف کهنه در مسیر زنده رد می‌شود؛ ۶۰ ردیف بین آموزش و آزمون پاک‌سازی می‌شود.
۶	ml_models.py	مدل با نسخه ناسازگار بارگذاری نمی‌شود؛ ستون‌ها اجباراً هم‌تراز می‌شوند؛ هندسه خروج در ریل‌های سخت محدود می‌شود؛ مدل ریسک وتوی سخت دارد.
۷	schemas.py	اهرم بالای ۱۰ از نظر ساختاری غیرممکن است؛ حد ضرر در سمت اشتباه رد می‌شود؛ احتمال‌ها نرمال می‌شوند.
۸	decision_engine.py	۱۰ قانون. هر رد شدن، دلیل دقیق ثبت می‌کند. قاعده جایگزین اجازه سفارش واقعی ندارد.
۹	executor.py	مارجین و اهرم قبل از هر سفارش تنظیم می‌شوند؛ سفارش محافظ فوراً ثبت می‌شود و اگر نشد پوزیشن بسته می‌شود؛ تریلینگ اول لغو بعد ثبت.
۱۰	risk_guard.py	افت روزانه، افت کل، زنجیره باخت، خرابی API، کلید دستی، قفل ماندگار روی دیسک.
۱۱	main.py	مسلح شدن پنج شرط دارد؛ تعویض موتور اول تخلیه می‌کند؛ چرخه همپوشان رد می‌شود؛ خاموشی مرتب است.

۲۶-۱. چهار جمله‌ای که در کد تکرار شده‌اند

🔴 **نکته:** «معامله روی کندل جعلی، بدتر از معامله نکردن است.» هرگز درون‌یابی نمی‌شود.

«یک پوزیشن اهرمی لخت، بدترین حالت ممکن است.» اگر حد ضرر ثبت نشد، پوزیشن بسته می‌شود.

«یک لحظه بدون حد ضرر امن‌تر از یک لحظه با دو حد ضرر است.» ترتیب لغو و ثبت.

«از دست دادن یک خط لاگ بد است، متوقف کردن حلقه معاملاتی بدتر.» صف دفتر هرگز بلوکه نمی‌کند.

۲۷. محدودیت‌ها و نقاط ضعف واقعی

یک گزارش کامل باید ضعف‌ها را هم بگوید. این‌ها را از خود کد و تاریخچه کامیت‌ها استخراج کرده‌ام.

موضوع	توضیح
سودآوری اثبات نشده	عدد خارج نمونه در README نرخ برد ۰٪ روی ۵ معامله است. نمونه خیلی کوچک است، ولی هیچ شهادی بر سودآوری هم وجود ندارد. مدل‌های آموزش‌دیده همراه پروژه نمی‌آیند.
ویژگی‌های ریزساختار بلااستفاده	دفتر سفارش، فاندینگ، بهره‌باز و لیکوئیدیشن جمع‌آوری و ذخیره می‌شوند ولی به مدل داده نمی‌شوند، چون بایننس تاریخچه عمیقی از آن‌ها نمی‌دهد. یعنی بخشی از داده‌ای که جمع می‌کنی، فقط تزئینی است.
پنل امن نیست	بدون توکن، هر کسی که به پورت ۸۰۰۰ برسد می‌تواند معامله را مسلح کند یا کلید قرمز بزند. نقاط فقط-خواندنی حتی با توکن هم باز می‌مانند. بدون HTTPS.
SQLite نویسنده تکی	با قفل درون‌پروژه‌ای حل شده، ولی اگر دو پروژه جدا همزمان بنویسند (مثلاً bootstrap در یک ترمینال و run در ترمینال دیگر)، قفل درون‌پروژه‌ای کمکی نمی‌کند.
عمق تاریخچه در عمل	هدف پیش‌فرض ۲ سال است، ولی بایننس همیشه آن‌قدر عمق نمی‌دهد. کد یک نشانگر "bootstrap_mode" "windowed-backfill" در گزارش می‌گذارد تا بشود تشخیص داد که «کد جدید اجرا شده ولی صرافی داده عمیق‌تری نداد» در برابر «کد قدیمی در حال اجراست».
تناقص جزئی در مستندات	در README و برخی توضیحات کد گاهی «۲۱ ویژگی» و گاهی «۲۶ ویژگی» ذکر شده. مقدار واقعی در FEATURE_COLUMNS برابر ۲۶ است.
وابستگی به کیفیت مدل	کل معماری فرض می‌کند مدل‌ها چیز مفیدی یاد گرفته‌اند. اگر مدل جهت فقط نویز یاد گرفته باشد، همه آن ۱۰ قانون فقط جلوی معاملات را می‌گیرند بدون اینکه لیه‌ای ایجاد کنند.
بدون تست خودکار	هیچ پوشه tests/ در مخزن وجود ندارد. درستی کد فقط با بازبینی و اجرای دستی تأیید می‌شود.
لغزش ساده‌سازی‌شده	لغزش یک عدد ثابت (۵ پایه) است، نه تابعی از عمق دفتر سفارش و اندازه سفارش. در بازار نازک، لغزش واقعی می‌تواند بسیار بیشتر باشد.

واژه	معنی
علّی (Causal)	محاسبه‌ای که فقط از داده‌های گذشته و حال استفاده می‌کند، نه آینده.
سوگیری نگاه به آینده	خطای استفاده از اطلاعاتی که در آن لحظه در دسترس نبوده. مرگبارترین اشتباه.
ایستایی (Stationarity)	ویژگی‌ای که معنایش در طول زمان ثابت بماند. «رتبه درصدی» ایستا است، «حجم خام» نیست.
MAD	میانۀ انحراف مطلق. معیار پراکندگی مقاوم در برابر داده‌های پرت.
ATR	میانگین محدوده واقعی — واحد سنجش نوسان یک ارز.
KAMA	میانگین متحرک تطبیقی کافمن که سرعتش را با شرایط بازار عوض می‌کند.
FDI	شاخص بُعد فراکتالی؛ می‌گوید بازار روند دارد یا درهم است.
GARCH	مدل آماری پیش‌بینی نوسان بر پایه خوشه‌ای بودن نوسان.
HMM	مدل مارکوف پنهان؛ حالت (رژیم) پنهان بازار را حدس می‌زند.
فیلتر رو به جلو (آلفا)	روش علّی تخمین حالت HMM که فقط از گذشته استفاده می‌کند.
سه‌مانعی	روش برجسب‌زنی با سه مانع: حد سود، حد ضرر، و انقضای زمانی.
MAE	بیشترین حرکت خلاف جهت در طول یک معامله — «درد مسیر».
MFE	بیشترین حرکت مطلوب در طول یک معامله.
پاک‌سازی (Purging)	حذف ردیف‌های مرزی بین آموزش و آزمون تا نشت برجسب رخ ندهد.
تقویت گرادیبانی	روش یادگیری ماشین با صدها درخت کوچک که خطای هم را اصلاح می‌کنند.
قاعده جایگزین (Heuristic)	یک قانون دستی ساده که وقتی مدل نیست جایش را می‌گیرد. با HEURISTIC مهر می‌شود.
مارجین ایزوله	هر معامله ودیعه خودش را دارد؛ ضررش به بقیه حساب سرایت نمی‌کند.
reduceOnly	سفارشی که فقط می‌تواند پوزیشن را کم کند، نه اینکه پوزیشن جدید باز کند.
قیمت مارک	قیمت منصفانه‌ای که صراف‌ی برای محاسبه لیکوئیدیشن استفاده می‌کند.
تریلینگ استاپ	حد ضرری که با حرکت قیمت به نفع تو، دنبالش می‌آید — ولی هرگز عقب نمی‌رود.
ایدمپوتنت	عملیاتی که تکرارش همان نتیجه یک‌بار اجرا را می‌دهد.
WAL	حالت «ثبت پیشاپیش» SQLite که خواندن و نوشتن همزمان را ممکن می‌کند.
Semaphore	شمارنده‌ای که تعداد عملیات همزمان را محدود می‌کند.
عقب‌نشینی نمایی	افزایش تصاعدی فاصله بین تلاش‌های مجدد: ۱، ۲، ۴، ۸ ثانیه...

۲۹. چک‌لیست راه‌اندازی و جمع‌بندی نهایی

۲۹-۱. چک‌لیست گام‌به‌گام

۱ محیط پایتون. پایتون ۳٫۱۱ یا بالاتر.

```
python -m venv .venv && source .venv/bin/activate  
pip install -r requirements.txt
```

۲ فایل تنظیمات.

```
cp .env.example .env
```

مهم‌ترین چیزی که باید عوض کنی: `UNIVERSE__REFERENCE_EQUITY` را روی *اندازه واقعی حساب خودت* بگذار. کل غریبال «تناسب با سرمایه کوچک» به این عدد وابسته است.

⚠ **هشدار:** در فایل `.env`، هرگز کامنت را جلوی یک مقدار خالی ننویس. `WEB__API_TOKEN= # my note` باعث می‌شود کل کامنت به عنوان مقدار توکن خوانده شود! کامنت را در خط جدا بنویس. (کامیت `05f399e` همین باگ را رفع کرده.)

۳ اجرا.

```
python main.py
```

سپس در مرورگر / آی‌پی‌سرور: `http://8000` را باز کن.

۴ **انتخاب ارزها.** به صفحه `/universe` برو. یا خودت تیک بزن یا دکمه «SUGGEST TOP 30» را بزن. بعد «SAVE & START» را بزن. `SETUP`.

🔴 **نکته:** اگر عمق تاریخچه ۲ سال برایت زیاد است (زمان و دیسک زیاد می‌خواهد)، `DATA__HISTORY_BOOTSTRAP_CANDLES` را کم کن: `105120 = یک سال`، `26280 = سه ماه`، `6000 ≈ ۲۱ روز`.

۵ **تماشای راه‌اندازی.** نوار پیشرفت داشبورد را ببین: `READY → TRAINING → COLLECTING_DATA`. گزارش آموزش ML را در همان صفحه بخوان و ببین هر مدل چه معیارهایی گرفته و کدام ویژگی‌ها مهم‌تر بوده‌اند.

۶ **شروع کاغذی.** دکمه `START PAPER`. حداقل چند هفته بگذار کار کند و صفحه `/audit` را بخوان. جالب‌ترین بخش، دیدن این است که چرا معامله نکرده.

۷ **قبل از پول واقعی.** سه شرط:

- `EXCHANGE__TESTNET=false` و کلیدهای واقعی با مجوز فیوچرز.
- هر چهار مدل واقعاً آموزش‌دیده باشند (وگرنه قانون R9 اجازه نمی‌دهد).

• ابتدا روی تست نت فیوچرز، بعد کاغذی، بعد واقعی با کوچک ترین اندازه ای که از حداقل های صرافی رد می شود.

امنیت. WEB__API_TOKEN را تنظیم کن و پورت ۸۰۰۰ را پشت فایروال یا تونل SSH بگذار.

۸

۲۹-۲. جمع بندی: کل پروژه در ده جمله

1. هر ۵ دقیقه (ثانیه ۱۰)، یک چرخه اجرا می شود.
2. برای ۳۰ ارزی که تو انتخاب کرده ای، داده تازه از بایننس گرفته می شود.
3. یک پلیس کیفیت داده را بررسی و در صورت نیاز تعمیر می کند؛ داده تعمیرناپذیر باعث حذف آن ارز در آن چرخه می شود.
4. از قیمت خام، ۲۶ ویژگی معنی دار ساخته می شود که هیچ کدام به آینده نگاه نمی کنند.
5. چهار مدل مستقل هر کدام یک نظر می دهند: کجا، کی، چگونه، چقدر.
6. یک موتور تصمیم بدون هوش مصنوعی، ۱۰ قانون سفت را چک می کند و اولین «نه» را ثبت می کند.
7. اگر همه چیز رد شد، یک سیگنال معاملاتی اعتبارسنجی شده ساخته می شود.
8. موتور اجرا (کاغذی یا واقعی) سفارش را می فرستد و بلافاصله حد سود و حد ضرر می گذارد.
9. یک نگهبان ریسک همه چیز را می باید و در صورت لزوم همه چیز را می بندد و قفل می کند.
10. هر تصمیم — حتی «معامله نکن» — با دلیل دقیقش در پایگاه داده ثبت می شود و در پنل وب قابل مشاهده است.



هشدار: یادآوری آخر. معامله اهرمی قراردادهای آتی دائمی، ریسک از دست دادن کل سرمایه را دارد. این نرم افزار برای اهداف پژوهشی و آموزشی ارائه شده، بدون هیچ ضمانتی و بدون هیچ ادعایی مبنی بر سودآوری. مسئولیت هر سرمایه ای که با آن به خطر می اندازی، با خودت است.